PI_Curs_03

3.0

Bine ați venit la Capitolul 3: **Rețele Neuronale Convoluționale (Convolutional Neural Networks - CNNs)**. Această lecție reprezintă o tranziție majoră de la rețelele neuronale generaliste (MLP) la arhitecturi specializate, optimizate pentru sarcini vizuale.

Ghid de Studiu Complet: 3. Rețele Neuronale Convoluționale (CNNs)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Acest capitol introduce Rețelele Neuronale Convoluționale (CNNs), o familie puternică de rețele neuronale concepute special pentru a gestiona datele de tip imagine. Lecția este crucială deoarece evidențiază limitările modelului anterior, MLP (Perceptronul Multistrat), în contextul imagistic: MLP-urile eșuează lamentabil deoarece **discipă structura spațială 2D a imaginii** prin aplatizarea acesteia într-un vector 1D, un mod nesatisfăcător de a trata datele bogate. CNN-urile rezolvă această problemă prin integrarea **cunoștințelor noastre apriorice** (prior knowledge) că **pixelii învecinați sunt tipic înrudiți**. Rezultatul este o arhitectură care este dominantă în viziunea computerizată și care este mult mai eficientă din punct de vedere computațional, deoarece necesită **mai puțini parametri** decât arhitecturile complet conectate.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Scopul CNN:** CNN-urile sunt concepute pentru a procesa datele vizuale (grilă bidimensională de pixeli) prin exploatarea structurii spațiale, spre deosebire de MLP-uri care o ignoră prin aplatizare.
2. **Eficiența Parametrilor:** CNN-urile sunt eficiente din punct de vedere computațional, deoarece necesită **mai puțini parametri** decât arhitecturile complet conectate (MLP).
3. **Ubicuite:** CNN-urile au devenit arhitecturi dominante și omniprezente (*ubiquitous*) în domeniul viziunii computerizate, fiind folosite în aplicații comerciale precum recunoașterea imaginilor, detecția obiectelor și segmentarea semantică.
4. **Componente Cheie:** "Coloana vertebrală (*backbone*)" a CNN-urilor este formată din operații de bază care includ **straturi convoluționale**, **padding**, **stride** și **straturi de *pooling***.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3. Rețele Neuronale Convoluționale

Structura Datelor de Intrare

- Pentru datele de tip imagine, fiecare exemplu constă dintr-o **grilă bidimensională de pixeli**.
- În funcție de tipul imaginii (alb-negru sau color), fiecare locație a pixelului poate fi asociată cu una sau mai multe valori numerice.

Limitările MLP-urilor

- O metodă de a aborda structura bogată a imaginii este de a ignora structura spațială și de a aplatiza imaginea într-un vector unidimensional, introducându-l într-un MLP complet conectat. Această metodă este considerată **nesatisfăcătoare (*unsatisfying*)**.
- Deoarece MLP-urile sunt invariante la ordinea caracteristicilor, s-ar obține rezultate similare chiar dacă coloanele matricei de design ar fi permuteate înainte de potrivirea parametrilor MLP-ului.

Motivația pentru CNN

- CNN-urile profită de **cunoștințele noastre apriorice** că **pixelii din apropiere sunt, de obicei, înrudiți între ei**.
- CNN-urile sunt o familie puternică de rețele neuronale concepute tocmai pentru acest scop.

Avantajele CNN

- CNN-urile sunt **ubicuite** în domeniul viziunii computerizate și sunt dominante în aplicații comerciale (recunoașterea imaginilor, detecția obiectelor, segmentarea semantică).
- Sunt **eficiente din punct de vedere computațional** și din punctul de vedere al eșantionului (*sample efficiency*).
- Necesită **mai puțini parametri** decât arhitecturile complet conectate.
- Operațiile de convoluție sunt ușor de **paralelizat** pe nucleele GPU.
- Designul CNN-urilor moderne se bazează pe inspirații din **biologie**, **teoria grupurilor** și experimentare.
- CNN-urile au fost adaptate cu succes chiar și pentru sarcini 1D (audio, text, serii de timp) și pe date structurate sub formă de grafuri sau în sisteme de recomandare.

Planul Capitolului (Structura Arhitecturală)

- Capitolul va prezenta operațiile de bază care formează "coloana vertebrală" a tuturor CNN-urilor:
  - Straturile convoluționale în sine.
  - Detalii precum **padding** (umplutura) și **stride** (pasul).

    - Straturile de **pooling** utilizate pentru a agrega informațiile din regiunile spațiale adiacente.
    - Utilizarea **canalelor multiple** la fiecare strat.
- Vor fi discutate exemple, inclusiv **LeNet**, prima rețea convoluțională de succes implementată.

Matematica unui Strat Convoluțional (Vedere preliminară)

- Formula de bază pentru calculul dimensiunii spațiale a ieșirii ($n'_h \times n'_w$) dintr-o operație de convoluție simplă (fără padding sau stride) este:

$$n'_h \times n'_w = (n_h - k_h + 1) \times (n_w - k_w + 1)$$

  Unde n_h, n_w sunt înălțimea/lățimea intrării, iar k_h, k_w sunt înălțimea/lățimea kernel-ului.
- Un strat convoluțional utilizează:
    - $\mathbf{X} \in \mathbb{R}^{C_{in} \times n_h \times n_w}$ (Input cu C_{in} canale de intrare).
    - $\mathbf{V} \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w}$ (Kernel-ul, care este un tensor cu C_{out} canale de ieșire).
    - $\mathbf{u} \in \mathbb{R}^{C_{out} \times 1}$ (Bias-ul, un scalar per canal de ieșire).
    - $\mathbf{H} \in \mathbb{R}^{C_{out} \times n'_h \times n'_w}$ (Ieșirea).

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Aplatizare (*Flattening*) vs. Structura Spațială

- **Explicație Simplificată (ELI5):** Imaginați-vă o imagine ca o grilă de joc de șah. Aplatizarea ar însemna să luăm toate pătrățelele și să le înșiruim pe o singură linie.
- *Asta se întâmplă deoarece:* Prin MLP, dacă schimbăm ordinea elementelor din acea linie (permute columns), modelul nu observă nicio diferență majoră. În realitate, relațiile dintre pătrățelele învecinate (ex: un colț întunecat și un altul luminos) sunt cruciale pentru a recunoaște forme. CNN-urile păstrează grila, iar operația de convoluție (kernelul) scanează local, păstrând și exploatând această informație spațială.

Concept Dificil: Eficiența Parametrilor (CNN vs. MLP)

- **Context:** De ce are un CNN mai puțini parametri decât un MLP?
- *Asta se întâmplă deoarece:* MLP-urile sunt **complet conectate** (Fully-connected). Dacă avem o imagine 100×100 (10.000 de intrări) și 1.000 de neuroni în primul strat ascuns, avem $10.000 \times 1.000 = 10$ milioane de ponderi. CNN-urile folosesc **ponderi partajate** (kernel-uri) și **conectivitate locală**. Un kernel de 5×5 are doar 25 de ponderi, care sunt aplicate pe întreaga imagine, reducând drastic numărul total de parametri care trebuie învățați.

LeNet (Context Istoric)

- Sursa menționează **LeNet** ca fiind prima rețea convoluțională implementată cu succes. Această rețea a fost dezvoltată de Yann LeCun în anii '90 pentru recunoașterea cifrelor scrise de mână (folosită pentru citirea cecurilor), demonstrând puterea CNN-urilor cu mult înainte de creșterea modernă a *deep learning*-ului.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Date de Intrare):**
  - **CAUZĂ:** Structura datelor de intrare este o grilă 2D de pixeli.
  - **EFECT:** MLP-urile eșuează prin distrugerea structurii spațiale $\implies$ CNN-urile sunt necesare pentru a profita de faptul că pixelii vecini sunt înrudiți.
- **Implicații Hardware:** Faptul că operațiile de convoluție sunt ușor de **paralelizat** pe GPU-uri a fost un factor major care a permis antrenarea eficientă a rețelelor CNN adânci pe hardware-ul modern.
- **Tranziția Abstracției:** Deși CNN-urile folosesc operații complexe, ele sunt concepute și implementate folosind conceptul de **bloc** (discutat în Lecția 2.8), permițând construirea rapidă a arhitecturilor adânci.

---

5. EXERCIȚII (Calcul pe Straturi Convoluționale)

Voi genera exerciții de calcul concentrate pe aspectele matematice ale unui strat convoluțional, folosind terminologia și formulele furnizate în surse.

Exercițiul C.4: Calculul Dimensiunii Ieșirii (Fără Padding/Stride)

Definiți un strat convoluțional simplu (fără *padding* și cu *stride* implicit 1).

1. Calculați dimensiunea spațială a ieșirii ($n'_h \times n'_w$) dacă imaginea de intrare are $n_h = 8$ și $n_w = 8$ (Intrare 8×8), iar kernel-ul (filtrul) are $k_h = 3$ și $k_w = 3$ (Kernel 3×3).
2. Calculați dimensiunea ieșirii dacă imaginea de intrare are $n_h = 6$ și $n_w = 6$ (Intrare 6×6), iar kernel-ul are $k_h = 4$ și $k_w = 4$ (Kernel 4×4).

Formula necesară (Conform surselor):

$$n'_h \times n'_w = (n_h - k_h + 1) \times (n_w - k_w + 1)$$

Rezolvare C.4:

1. **Cazul 1 (Intrare 8×8, Kernel 3×3):**

$$n'_h = (8 - 3 + 1) = 6$$

$$n'_w = (8 - 3 + 1) = 6$$

Dimensiunea ieșirii este $\mathbf{6} \times \mathbf{6}$.

2. **Cazul 2 (Intrare 6×6, Kernel 4×4):**

$$n'_h = (6 - 4 + 1) = 3$$

$$n'_w = (6 - 4 + 1) = 3$$

Dimensiunea ieșirii este $\mathbf{3} \times \mathbf{3}$.

Exercițiul C.5: Calculul Numărului Total de Parametri (Greutăți și Bias-uri)

Definiți un strat convoluțional (FC) cu:

- Număr de canale de intrare $C_{in} = 5$.
- Număr de canale de ieșire $C_{out} = 10$.
- Dimensiunea kernel-ului $k_h \times k_w = 4 \times 4$.

Calculați numărul total de parametri reali (greutăți V și bias-uri u) pentru acest strat.

Formule necesare (Derivate din surse):

- Numărul de greutăți V: $C_{out} \times C_{in} \times k_h \times k_w$
- Numărul de bias-uri u: C_{out}

Rezolvare C.5:

1. **Calculul Numărului de Greutăți (V):**

   V = 10 \text{ (C_{out})} \times 5 \text{ (C_{in})} \times 4 \text{ (k_h)} \times 4 \text{ (k_w)} = 800

2. **Calculul Numărului de Bias-uri (u):** Deoarece există un scalar bias $\mathbf{u}$ pentru fiecare canal de ieșire, avem:

$$u = C_{out} = 10$$

3. **Total Parametri:**

$$\text{Total parametri} = 800 + 10 = \mathbf{810}$$

Exercițiul C.6: Calculul unei Activări (Punctual)

Considerăm un strat convoluțional cu un singur canal de intrare ($C_{in} = 1$) și un singur canal de ieșire ($C_{out} = 1$). Kernel-ul ($k_h \times k_w$) este de 2×2. Presupunem că bias-ul (u) este 1.

Să presupunem că Kernel-ul și o porțiune din matricea de intrare X sunt:

$$V = [1 \quad -1\ 0 \quad 2], \quad \text{Porțiune } X \text{ (aplicată la } V) = [3 \quad 4\ 1 \quad 0]$$

Calculați valoarea de ieșire (activarea ne-neliniară) $H_{i,j,1}$ la această poziție.

Formula necesară (Conform surselor, Ecuația 10):

$$H_{i,j,d} = u_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c} [V]_{a,b,c,d} [X]_{i+a,j+b,c}$$

Simplificată pentru $C_{in} = 1, C_{out} = 1$ și $k_h = k_w = 2$ (unde Δ ar fi 0):

$$H_{i,j} = u + \sum_{r=0}^{1} \sum_{s=0}^{1} V_{r,s} X_{i+r,j+s}$$

Rezolvare C.6:

$$H_{i,j} = u + (V_{11}X_{11} + V_{12}X_{12} + V_{21}X_{21} + V_{22}X_{22})$$

$$H_{i,j} = 1 + (1 \times 3) + (-1 \times 4) + (0 \times 1) + (2 \times 0)$$

$$H_{i,j} = 1 + 3 - 4 + 0 + 0$$

$$H_{i,j} = \mathbf{0}$$

3.1

Această lecție, **3.1 De la straturi complet conectate la convoluții**, marchează trecerea de la modelele generice (MLP) la cele optimizate pentru date structurate spațial, în special imagini. Înțelegerea acestei tranziții este fundamentală, deoarece explică de ce CNN-urile sunt arhitectura dominantă în viziunea computerizată.

Ghid de Studiu Complet: 3.1 De la straturi complet conectate la convoluții

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Această lecție stabilește motivația critică pentru utilizarea Rețelelor Neuronale Convoluționale (CNN) în locul Perceptronilor Multistrat (MLP) pentru sarcini de viziune computerizată. Punctul de plecare este recunoașterea faptului că, deși MLP-urile sunt adecvate pentru date tabulare (structurate pe rânduri și coloane), ele sunt complet inadecvate pentru date perceptive de înaltă dimensiune, cum ar fi imaginile. Motivul principal este **costul de parametrizare insuportabil** al MLP-urilor (miliarde de parametri pentru imagini de înaltă rezoluție). CNN-urile rezolvă această problemă prin exploatarea structurii bogate a imaginii și prin introducerea a două principii de design esențiale: **Invarianța la Translație (*Translation Invariance*)** și **Principiul Localității (*Locality Principle*)**. Aceste principii permit **partajarea ponderilor** (transformând straturile complet conectate în straturi convoluționale) și reducând drastic numărul de parametri la câteva sute, în loc de miliarde.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Eșecul MLP:** Un singur strat complet conectat într-un MLP necesită miliarde (10^9) de parametri pentru a procesa o imagine de un megapixel (10^6 dimensiuni de intrare).
2. **Principiul Invarianței la Translație:** Un detector (kernel) ar trebui să răspundă similar la aceeași caracteristică, indiferent de locația ei în imagine. Acest lucru duce la **partajarea ponderilor** (kernelul $\mathbf{V}$ nu depinde de locația (i, j)).
3. **Principiul Localității:** Straturile timpurii ar trebui să se concentreze doar pe **regiunile locale** ale imaginii, fără a ține cont de conținutul din regiunile îndepărtate.
4. **Reducerea Drastică a Parametrilor:** Prin aplicarea invarianței la translație și localității, trecem de la tenzori de ordinul patru $\mathbf{W}$ (în MLP) la un **kernel de convoluție** $\mathbf{V}$ mult mai mic, reducând costul de la miliarde la sute de parametri.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.1 De la straturi complet conectate la convoluții

- **Aplicabilitatea MLP:** Modelele discutate anterior rămân adecvate pentru lucrul cu **date tabulare** (rânduri = exemple, coloane = caracteristici). Pentru astfel de date, de multe ori nu există o structură *a priori* de asumat cu privire la interacțiunea caracteristicilor.
- **Provocarea Datelor Perceptive:** Pentru date perceptive de înaltă dimensiune (ex: imagini), rețelele fără structură (MLP-urile) nu sunt potrivite.
- **Costul Parametrizării MLP:**
  - Dacă o imagine are o rezoluție de un megapixel (10^6 dimensiuni de intrare).
  - Chiar și cu o reducere agresivă la 1000 de unități ascunse, un singur strat complet conectat ar necesita $10^6 \times 10^3 = \mathbf{10^9}$ parametri.
  - Învățarea acestui număr masiv de parametri ar fi nerealizabilă (*infeasible*).
- **Exploatarea Structurii:** CNN-urile (Rețele Neuronale Convoluționale) sunt o soluție creativă care exploatează structura bogată și cunoscută existentă în imaginile naturale.

Inspirația din Invarianța Spațială

- **Exemplul "Unde este Waldo?":** Căutarea unui obiect (Waldo) nu ar trebui să fie excesiv de preocupată de locația sa precisă.
- Un detector pentru Waldo ar trebui să funcționeze indiferent **unde** este localizat Waldo.
- CNN-urile sistematizează această idee de **invarianță spațială (*spatial invariance*)**, exploatând-o pentru a învăța reprezentări utile cu **mai puțini parametri**.

Principiile de Design pentru CNN-uri (Desiderata)

1. **Invarianța la Translație (*Translation Invariance*):** Straturile timpurii trebuie să răspundă similar la aceeași porțiune din imagine, indiferent unde apare ea.
2. **Principiul Localității (*Locality Principle*):** Straturile timpurii ar trebui să se concentreze doar pe **regiuni locale** ale imaginii, fără a ține cont de conținutul din regiunile

îndepărtate.

Formalizarea Matematică (De la MLP la Convoluție)

- **Tenzori:** Sunt o structură generică de date (array-uri n-dimensionale) care generalizează scalarii, vectorii (tenzori de ordinul întâi) și matricile (tenzori de ordinul doi).
- **MLP pentru Imagini (Punct de plecare):** Pentru a trata imaginile $\mathbf{X}$ (matrici 2D) și reprezentările ascunse $\mathbf{H}$ ca având structură spațială, ponderile $\mathbf{W}$ ale unui strat complet conectat devin **tenzori de ordinul patru**.
- **Aplicarea Invarianței la Translație:**
  - Pentru ca o schimbare în intrarea $\mathbf{X}$ să ducă doar la o schimbare (shift) în reprezentarea ascunsă $\mathbf{H}$, este necesar ca ponderile și bias-urile să nu depindă de locația (i, j).
  - Astfel, $[\mathbf{V}]_{i,j,a,b}$ devine simplu $[\mathbf{V}]_{a,b}$, iar bias-ul $_{i,j}$ devine o constantă scalară u.
  - Această simplificare duce la forma unei convoluții:

$$[\mathbf{H}]_{i,j} = u + \sum_a \sum_b [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b}$$

  - **Progres:** $\mathbf{V}_{a,b}$ are mult mai puțini coeficienți decât $\mathbf{V}_{i,j,a,b}$, ceea ce reprezintă un progres semnificativ.
- **Aplicarea Localității:**
  - Se crede că nu este necesar să se privească foarte departe de locația (i, j) pentru a aduna informații relevante.
  - Aceasta înseamnă că $\mathbf{V}_{a,b} = 0$ în afara unui anumit interval $|| > \Delta$ sau $|\mathbf{b}| > \Delta$.
  - Formula devine:

$$[\mathbf{H}]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [\mathbf{V}]_{a,b} [\mathbf{X}]_{i+a,j+b} \quad (1)$$

  - Această ecuație (1) descrie un **strat convoluțional**. $\mathbf{V}$ este numit **kernel de convoluție** sau filtru.

Canale Multiple

- Imaginile sunt de fapt **tenzori de ordinul trei** (înălțime, lățime, canal - ex: RGB).
- Pentru a gestiona canale multiple de intrare (C_{in} indexat de c) și a produce canale multiple de ieșire (C_{out} indexat de d), kernelul $\mathbf{V}$ necesită o a patra coordonată $\mathbf{V}[a, b, c, d]$.
- Reprezentările ascunse $\mathbf{H}$ sunt, de asemenea, tenzori de ordinul trei (numite **feature maps** sau canale).
- Bias-ul devine un vector $\mathbf{u}$ (un scalar pentru fiecare canal de ieșire).

- **Formula Generală (pentru canale multiple):**

$$[\mathbf{H}]_{i,j,d} = [\mathbf{u}]_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c} [\mathbf{V}]_{a,b,c,d} [\mathbf{X}]_{i+a,j+b,c} \quad (3)$$

Convoluție vs. Cross-Correlation

- Ecuația (1) și (3) descriu mai corect o operație de **cross-correlation**.
- Definiția matematică formală a convoluției implică o inversare (flipping) a uneia dintre funcții, folosind $(i-a, j-b)$ în loc de $(i+a, j+b)$.
- Totuși, distincția este în mare parte cosmetică, deoarece ponderile kernelului $\mathbf{V}$ sunt parametri care pot fi învățați, iar notarea poate fi ajustată.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Concept Dificil: Invarianța la Translație și Partajarea Ponderilor

- **Explicație Simplificată:** În MLP, fiecare pixel din imagine este conectat la fiecare neuron ascuns printr-o pondere unică. Dacă mutați o pisică cu un pixel la dreapta, trebuie să învățați un set complet nou de ponderi.
- **CNN (Invarianța):** CNN-ul folosește un singur "detector" (kernelul $\mathbf{V}$) pentru a scana întreaga imagine. Dacă kernelul detectează un ochi într-un colț, același kernel (aceleași ponderi) va detecta ochiul și în centrul imaginii. Acesta este mecanismul de **partajare a ponderilor**, care reduce numărul de parametri de la ordinul miliardelor la ordinul sutelor.

Concept Dificil: Tenzorii de Ordinul Trei și Canalele

- **Analogie:** Gândiți-vă la un set de diapozitive stivuite. O imagine color este formată din trei diapozitive stivuite (roșu, verde, albastru). Acesta este un tenzor de ordinul trei.
- **Canalele de Ieșire (Feature Maps):** După ce un strat convoluțional procesează intrarea, el creează noi seturi de diapozitive (canale de ieșire sau feature maps). Intuitiv, unele canale se pot specializa să recunoască marginile, în timp ce altele recunosc texturile.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Parametrizare vs. Structură):**
  - **CAUZĂ:** Structura MLP (complet conectată) duce la un număr de parametri insuportabil pentru datele de înaltă dimensiune.
  - **EFECT:** Introducerea principiilor Invarianței la Translație și Localității forțează modelul să utilizeze **conectivitate locală** și **partajarea ponderilor**, transformând

stratul MLP într-un strat convoluțional care este realizabil din punct de vedere computațional.

- **Conexiunea cu Optimizarea:** Reducerea drastică a numărului de parametri (de la miliarde la sute) ușurează semnificativ sarcina de colectare a datelor și previne *overfitting*-ul, deoarece necesită un set de date mult mai mic pentru a învăța parametrii.

---

5. EXERCITII (Calcul pe Straturi Convoluționale)

Vom genera un exercițiu detaliat de calcul pentru un strat convoluțional, urmând modelul de rezolvare complex prezentat în sursa *03_11_Exercises.pdf*.

Exercițiul C.7: Calculul Ieșirii și Costul Parametrilor (Strat Convoluțional 2D)

Definiți un strat convoluțional cu:

- Canale de intrare $C_{in} = 2$.
- Canale de ieșire $C_{out} = 2$.
- Dimensiunea kernel-ului $k_h \times k_w = 2 \times 2$.
- Imagine de intrare $n_h \times n_w = 4 \times 4$.
- Fără *padding* și *stride* implicit (1).

A. Calculați dimensiunea spațială a ieșirii. B. Specificați numărul total de parametri reali (greutăți V și bias-uri u) pentru strat. C. Calculați un element specific al ieșirii $[\mathbf{H}]_{1,1,1}$ (Elementul din rândul 1, coloana 1, canalul de ieșire 1), dat fiind următoarele valori (simplificate):

$$\mathbf{u} = [1\ 0] \in \mathbb{R}^{2\times 1}$$

Pentru **Canalul de Ieșire 1** ($d = 1$):

$$V_{,,c=1,d=1} = [1 \quad 0\ 0 \quad 1], \quad V_{,,c=2,d=1} = [-1 \quad 0\ 0 \quad 0]$$

Și următoarea porțiune de intrare $\mathbf{X}$ (pentru rândul 1, coloana 1, 2×2 patch):

$$\text{analul de Intrare 1: } X_{\text{patc},c=1} = [5 \quad 6\ 2 \quad 1]$$

$$\text{analul de Intrare 2: } X_{\text{patc},c=2} = [1 \quad 0\ 0 \quad 3]$$

Formula necesară (simplificată pentru $i = 1, j = 1, d = 1$):

$$[\mathbf{H}]_{1,1,1} = [\mathbf{u}]_1 + \sum_{c=1}^{2} \left(\sum_{a=0}^{1} \sum_{b=0}^{1} [\mathbf{V}]_{a,b,c,1} [\mathbf{X}]_{1+a,1+b,c} \right)$$

(Nota: Sumarea este pe $a, b \in 0, 1$ pentru kernel 2×2).

Rezolvare C.7:

A. Calculul Dimensiunii Spațiale a Ieșirii ($n'_h \times n'_w$):

$$n'_h \times n'_w = (n_h - k_h + 1) \times (n_w - k_w + 1)$$

$$n'_h \times n'_w = (4 - 2 + 1) \times (4 - 2 + 1) = 3 \times 3$$

Dimensiunea spațială a ieșirii este $\mathbf{3} \times \mathbf{3}$. Ieșirea totală $\mathbf{H}$ este un tensor $2 \times 3 \times 3$.

B. Calculul Numărului Total de Parametri:

$$\text{Număr greutăți } V = C_{out} \times C_{in} \times k_h \times k_w = 2 \times 2 \times 2 \times 2 = 16$$

$$\text{Număr bias-uri } u = C_{out} = 2$$

$$\text{Total parametri} = 16 + 2 = \mathbf{18}$$

C. Calculul elementului $[\mathbf{H}]_{1,1,1}$:

Pasul 1: Contribuția din Canalul de Intrare 1 ($c = 1$):

$$\text{Produs } = V_{,,c=1,d=1} \ X_{\text{patc},c=1}$$

$$\text{Suma } = (1 \times 5) + (0 \times 6) + (0 \times 2) + (1 \times 1) = 5 + 0 + 0 + 1 = 6$$

Pasul 2: Contribuția din Canalul de Intrare 2 ($c = 2$):

$$\text{Produs } = V_{,,c=2,d=1} \ X_{\text{patc},c=2}$$

$$\text{Suma } = (-1 \times 1) + (0 \times 0) + (0 \times 0) + (0 \times 3) = -1 + 0 + 0 + 0 = -1$$

Pasul 3: Adăugarea Bias-ului și Sumarea Contribuțiilor: Bias-ul pentru canalul $d = 1$ este $[\mathbf{u}]_1 = 1$.

$$[\mathbf{H}]_{1,1,1} = [\mathbf{u}]_1 + (\text{Suma } c = 1) + (\text{Suma } c = 2)$$

$$[\mathbf{H}]_{1,1,1} = 1 + 6 + (-1) = \mathbf{6}$$

3.2

Bine ați venit! Acum că am stabilit motivația teoretică pentru Rețelele Neuronale Convoluționale (CNNs), lecția 3.2 ne mută la implementarea și înțelegerea operației de bază: **Convoluțiile pentru Imagini**. Deși numele este "convoluție", vom vedea că operația reală este mai exact **cross-correlation**.

Ghid de Studiu Complet: 3.2 Convoluții pentru Imagini

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.2 se concentrează pe aplicarea practică a straturilor convoluționale, păstrând imaginile ca exemplu principal de date structurate. Ideea centrală este că un strat convoluțional combină un **tensor de intrare** cu un **tensor kernel** pentru a produce un

tensor de ieșire printr-o operație de **cross-correlation**. Deși, strict vorbind, "convoluția" este un termen inexact (o greșeală de nume sau *misnomer*) pentru operația folosită, termenul "convolution" este menținut în standardele literaturii de *deep learning*. Înțelegerea acestei operații de cross-correlation, modul în care afectează dimensiunea ieșirii și cum se leagă de conceptul de **câmp receptiv (*receptive field*)**, este esențială pentru a construi rețele neuronale adânci eficiente.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Cross-Correlation vs. Convoluție:** Operația efectuată de straturile convoluționale este, din punct de vedere matematic, mai precis descrisă ca **cross-correlation**.
2. **Mecanismul de Operare:** Operația implică o **fereastră de convoluție (*kernel window*)** care este poziționată în colțul superior-stâng al tensorului de intrare, glisează (slides) peste imagine, de la stânga la dreapta și de sus în jos.
3. **Parametrii Stratului:** Cei doi parametri ai unui strat convoluțional sunt **kernelul** și **bias-ul scalar**.
4. **Reducerea Dimensiunii Ieșirii:** Atunci când kernelul are lățimea și înălțimea mai mare decât unu, dimensiunea ieșirii este **mai mică** decât dimensiunea intrării.
5. **Câmpul Receptiv (*Receptive Field*):** Câmpul receptiv al unui element de ieșire reprezintă toate elementele din straturile anterioare care îi afectează calculul.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.2 Convoluții pentru Imagini

Terminologie și Operația de Bază

- Deși înțelegem funcționarea straturilor convoluționale în teorie, trebuie să vedem cum funcționează în practică.
- Rețelele convoluționale sunt arhitecturi eficiente pentru explorarea structurii în datele de tip imagine.
- Operațiile pe care straturile convoluționale le exprimă sunt, strict vorbind, mai precis descrise ca **cross-correlations**.
- În cadrul unui strat convoluțional, un **tensor de intrare** și un **tensor kernel** sunt combinate pentru a produce un **tensor de ieșire** printr-o operație de cross-correlation.
- Un strat convoluțional aplică cross-correlation între intrare și kernel și adaugă un **bias scalar** pentru a produce o ieșire.
- Cei doi parametri ai unui strat convoluțional sunt **kernelul** și **bias-ul scalar**.

Mecanismul Cross-Correlation 2D (Fără Canale)

- Dacă ignorăm canalele, intrarea este un tensor bidimensional.

- Exemplu: Intrarea este un tensor 3×3, iar kernelul (sau **fereastra de convoluție** - *convolution window*) are înălțimea și lățimea 2 (adică 2×2).
- În operația de cross-correlation bidimensională, fereastra de convoluție este poziționată inițial în **colțul superior-stâng** al tensorului de intrare.
- Fereastra **glisează** peste tensorul de intrare, de la stânga la dreapta și de sus în jos.
- La fiecare poziție, sub-tensorul de intrare din fereastră și tensorul kernel sunt **înmulțite element cu element** (*element-wise*) și apoi rezultatul este **însumat**, producând o valoare scalară unică.
- Această valoare scalară dă valoarea tensorului de ieșire la locația corespunzătoare.

Dimensiunea ieșirii

- În exemplul 3×3 Input cu 2×2 Kernel, ieșirea are o înălțime de 2 și o lățime de 2 (Output 2×2).
- De-a lungul fiecărei axe, **dimensiunea ieșirii este ușor mai mică** decât dimensiunea intrării.
- Formula pentru dimensiunea ieșirii ($n'_h \times n'_w$) când kernelul are înălțimea k_h și lățimea k_w este:

$$(n_h - k_h + 1) \times (n_w - k_w + 1)$$

- Această reducere apare deoarece cross-correlation poate fi calculată doar pentru locațiile în care kernelul se potrivește **în întregime** în interiorul imaginii.
- Pentru a menține dimensiunea neschimbată, se poate folosi **padding** (umplerea imaginii cu zerouri în jurul marginii).

Convoluție Strictă vs. Cross-Correlation (Clarificare Matematică)

- Dacă un strat ar efectua operația de convoluție strictă, ar fi necesară **inversarea (flipping)** kernelului bidimensional (orizontal și vertical) și apoi efectuarea operației de cross-correlation cu tensorul de intrare.
- Deoarece kernelii sunt **învățați din date** în *deep learning*, ieșirile straturilor convoluționale rămân neafectate, indiferent dacă se efectuează operația strictă de convoluție sau operația de cross-correlation.
- Prin urmare, în literatura de *deep learning*, se continuă referirea la operația de cross-correlation ca fiind o **convoluție**.

Câmpul Receptiv (*Receptive Field*)

- Ieșirea stratului convoluțional (numită **feature map**) poate fi privită ca reprezentări învățate (caracteristici) în dimensiunile spațiale.
- **Câmpul receptiv** al unui element x dintr-un strat se referă la **toate elementele** (din toate straturile anterioare) care pot afecta calculul lui x în timpul propagării înainte.
- Pentru un kernel 2×2 în primul strat, câmpul receptiv al elementului de ieșire (de ex., valoarea 19 din Figura 3) este format din cele patru elemente umbrite din intrare.

- Câmpul receptiv poate fi **mai mare** decât dimensiunea reală a intrării.
- Prin construirea unei rețele mai adânci, se poate obține un câmp receptiv mai mare pentru a detecta caracteristici de intrare pe o zonă mai largă. De exemplu, un element z calculat dintr-o a doua convoluție 2×2 (luând ieșirea 2×2, denotată $\mathbf{Y}$, ca intrare) include toate cele nouă elemente de intrare inițiale în câmpul său receptiv.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Kernelul (Filtru) și Bias-ul Scalar

- **Explicație Simplificată:** Kernelul este ca o **lupă de detectare a tiparelor**. Este o mică grilă de numere (greutăți) pe care modelul o învață. La fiecare poziție, el verifică dacă porțiunea din imagine se potrivește cu tiparul său.
- **Bias-ul Scalar:** Bias-ul este un număr constant adăugat la **toate** ieșirile din acel canal (feature map). Dacă un strat produce 10 canale de ieșire, el are 10 bias-uri, câte unul pentru fiecare canal de ieșire.

Concept Dificil: Câmpul Receptiv (*Receptive Field*)

- **Analogie (Vederea Umană):** Câmpul receptiv al unui neuron este ca o bucată de imagine pe care un anumit grup de celule neuronale din creier o "vede" sau o procesează.
  - **Straturile Timpurii (Câmp Receptiv Mic):** Sunt ca niște celule care văd doar pixeli mici sau margini locale. (Ex: Un element de ieșire vede doar o regiune 2×2 din imaginea inițială).
  - **Straturile Adânci (Câmp Receptiv Mare):** Prin compunerea convoluțiilor, neuronii din straturile mai adânci "văd" o zonă mult mai mare, combinând informațiile locale pentru a detecta obiecte complexe (Ex: combină muchii pentru a detecta un ochi, apoi combină ochii pentru a detecta o față). Creșterea profunzimii rețelei crește câmpul receptiv.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu Propagarea Înainte (Lecția 2.6):** Operația de cross-correlation este pasul fundamental de calcul (*forward pass*) în straturile convoluționale, înlocuind operația matricială masivă XW din MLP-uri.
- **Conexiunea cu Inițializarea (Lecția 2.7):** La fel ca ponderile din straturile complet conectate, kernelii (care sunt parametrii stratului convoluțional) sunt inițializați aleatoriu la antrenare.

- **Implicația asupra Obiectivelor de Învățare:** Deși putem concepe kerneluri manuale (ex: $[1, -1]$ pentru detectarea muchiilor), în *deep learning* **trebuie să învățăm** acești kerneluri din date, deoarece devine imposibil de specificat manual ce ar trebui să facă fiecare filtru în straturile succesive de convoluție.

---

5. EXERCITII (Calcul pe Straturi Convoluționale)

Vom genera exerciții bazate pe structura datelor, dimensiunea ieșirii și operația de cross-correlation 2D, urmând modelul de calcul numeric din surse.

Exercițiul C.8: Calculul Ieșirii (Cross-Correlation 2D, Fără Canale)

Considerăm o singură operație de cross-correlation bidimensională (fără bias).

- **Intrare (X):** 4×4
- **Kernel (V):** 3×3

$$X = [1 \quad 1 \quad 0 \quad 0\,0 \quad 2 \quad 1 \quad 0\,3 \quad 4 \quad 5 \quad 6\,0 \quad 0 \quad 1 \quad 1], \quad V = [1 \quad 0 \quad 1\,0 \quad 1 \quad 0\,1 \quad 0 \quad 1]$$

1. Calculați dimensiunea spațială a ieșirii ($n'_h \times n'_w$).
2. Calculați valoarea elementului de ieșire din colțul superior-stâng, $\mathbf{H}_{0,0}$.

Rezolvare C.8:

1. **Dimensiunea Ieșirii:**
   - Intrare $n_h = 4, n_w = 4$. Kernel $k_h = 3, k_w = 3$.
   - $n'_h \times n'_w = (n_h - k_h + 1) \times (n_w - k_w + 1)$.
   - $n'_h \times n'_w = (4 - 3 + 1) \times (4 - 3 + 1) = 2 \times 2$.
   - Dimensiunea ieșirii este $\mathbf{2 \times 2}$.
2. **Calculul $\mathbf{H}_{0,0}$ (Colțul Superior-Stâng):** Kernelul V este aplicat pe sub-matricea de intrare din colțul superior-stâng ($X_{0:3,0:3}$):

$$X_{\text{patc}} = [1 \quad 1 \quad 0\,0 \quad 2 \quad 1\,3 \quad 4 \quad 5]$$

$$\mathbf{H}_{0,0} = \sum_{i,j} V_{i,j} X_{\text{patc},i,j}$$

$$\mathbf{H}_{0,0} = (1 \cdot 1) + (0 \cdot 1) + (1 \cdot 0) + (0 \cdot 0) + (1 \cdot 2) + (0 \cdot 1) + (1 \cdot 3) + (0 \cdot 4) + (1 \cdot 5)$$

$$\mathbf{H}_{0,0} = 1 + 0 + 0 + 0 + 2 + 0 + 3 + 0 + 5 = \mathbf{11}$$

Exercițiul C.9: Calculul Total al Parametrilor și Relația cu Bias-ul

Definiți un strat convoluțional cu:

- Canale de intrare $C_{in} = 4$.
- Canale de ieșire $C_{out} = 8$.
- Dimensiunea kernel-ului $k_h \times k_w = 5 \times 5$.

1. Calculați numărul total de parametri reali (greutăți V și bias-uri u) ai stratului.
2. Dacă bias-ul scalar aplicat canalului de ieșire 1 este $u_1 = 3$, la câte elemente din tensorul de ieșire este adăugată această valoare?

Rezolvare C.9:

1. **Calculul Total al Parametrilor:**
   - Număr greutăți V: $C_{out} \times C_{in} \times k_h \times k_w = 8 \times 4 \times 5 \times 5 = 800$.
   - Număr bias-uri u: $C_{out} = 8$.
   - Total parametri: $800 + 8 = \mathbf{808}$.
2. **Aplicarea Bias-ului:** Bias-ul scalar u_1 este adăugat **tuturor elementelor** canalului de ieșire 1. Dacă dimensiunea spațială a ieșirii este $n'_h \times n'_w$, valoarea u_1 este adăugată la $n'_h \times n'_w$ elemente. (Bias-ul este constant pentru un întreg *feature map*).

Exercițiul C.10: Extinderea Câmpului Receptiv (Layer Stacking)

Un MLP folosește două straturi convoluționale consecutive, ambele cu kernel 3×3, fără *padding* și *stride* 1.

1. Dacă un element de ieșire din primul strat (FC1) are un câmp receptiv 3×3 în imaginea de intrare inițială, care este dimensiunea câmpului receptiv în imaginea de intrare inițială pentru un element de ieșire din cel de-al doilea strat (FC2)? *(Indiciu: FC2 ia ieșirea $n'_h \times n'_w$ a lui FC1 ca intrare.)*

Rezolvare C.10:

1. **Câmpul Receptiv Final:**
   - **Stratul 1 (FC1):** Kernel 3×3. Câmpul receptiv (RF1) este 3×3.
   - **Stratul 2 (FC2):** Kernel 3×3. Un element de ieșire din FC2 combină o regiune 3×3 din ieșirea FC1. Deoarece fiecare element din ieșirea FC1 "vede" o regiune 3×3 din intrarea originală, elementul final din FC2 "vede" o regiune extinsă de:

     $$\text{RF total} = (k_h^{(1)} + k_h^{(2)} - 1) \times (k_w^{(1)} + k_w^{(2)} - 1)$$

     $$\text{RF total} = (3 + 3 - 1) \times (3 + 3 - 1) = 5 \times 5$$

     Câmpul receptiv pentru un element din FC2 este de 5×5 în imaginea de intrare originală.

3.3

Am ajuns la o lecție critică în proiectarea CNN-urilor: **3.3 Padding și Stride**. Aceste două concepte sunt esențiale deoarece ele ne permit să controlăm precis dimensiunea ieșirii

(*output shape*) a unui strat convoluțional, prevenind fie pierderea de informații, fie creșterea necontrolată a adâncimii rețelei.

Ghid de Studiu Complet: 3.3 Padding și Stride

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.3 abordează o problemă inerentă a operației de convoluție (cross-correlation): **reducerea dimensiunii spațiale a ieșirii**. Deoarece kernelii au, în general, înălțimea și lățimea mai mare decât 1, după aplicarea convoluției, ieșirile tind să fie considerabil mai mici decât intrarea. Aplicarea multor straturi succesive de convoluție duce la "pierderea de pixeli de pe perimetru", ignorând orice informație interesantă de pe granițele imaginii originale. Pentru a rezolva această problemă, au fost introduse două tehnici standard: **Padding** (pentru a preveni pierderea dimensiunii) și **Stride** (pentru a reduce dimensiunea drastic, în scopuri de eficiență sau *downsampling*).

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Problema:** Deoarece kernelul trebuie să se potrivească în întregime în imagine, aplicarea convoluțiilor reduce dimensiunea spațială a ieșirii. Straturile succesive acumulează această reducere.
2. **Padding (Soluția la Pierderea Informației): Padding-ul** este cea mai populară tehnică pentru a gestiona pierderea de informații de pe margini. Acesta adaugă pixeli suplimentari (de obicei, cu valoarea zero) în jurul perimetrului imaginii de intrare, crescând dimensiunea efectivă.
3. **Conservarea Dimensiunii:** Putem seta padding-ul înălțimii (p_h) și lățimii (p_w) astfel încât ieșirea să aibă **aceeași înălțime și lățime ca intrarea**, adesea prin setarea $p_h = k_h - 1$ și $p_w = k_w - 1$.
4. **Stride (Soluția la Reducerea Drastică): Stride** (pasul) se referă la numărul de rânduri și coloane parcurse la fiecare glisare a ferestrei de convoluție. Stride-ul implicit este 1, dar un *stride* mai mare de 1 este folosit pentru **eficiență computațională** sau pentru a **reduce drastic dimensionalitatea** (*downsample*).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.3 Padding și Stride

Reducerea Dimensiunii Ieșirii (Motivația)

- În exemplul anterior (Secțiunea 3.2), o intrare de 3×3 cu un kernel de convoluție de 2×2 produce o ieșire de 2×2.

- În general, pentru o intrare de $n_h \times n_w$ și un kernel de $k_h \times k_w$, dimensiunea ieșirii este:

$$(n_h - k_h + 1) \times (n_w - k_w + 1)$$

- Deoarece kernelii sunt, în general, mai mari decât 1×1, ieșirile sunt considerabil mai mici decât intrarea.
- Dacă începem cu o imagine 240×240 și aplicăm 10 straturi succesive de convoluții 5×5, imaginea se reduce la 200×200, "tăind" 30% din imagine.

Padding (Umplutura)

- O problemă majoră este **pierderea de pixeli pe perimetrul imaginii**. Deși se pierd doar câțiva pixeli per convoluție, acest lucru se acumulează rapid în rețelele adânci.
- **Soluția:** Adăugarea de pixeli suplimentari în jurul limitei imaginii de intrare.
- Acești pixeli suplimentari sunt setați tipic la **zero** (**Zero Padding**).
- **Obiectivul 1 (Păstrarea Dimensiunii):** În multe cazuri, se dorește ca intrarea și ieșirea să aibă aceeași înălțime și lățime, facilitând construcția rețelelor adânci.
- Pentru a obține aceeași dimensiune (cu *stride* 1), se setează $p_h = k_h - 1$ și $p_w = k_w - 1$.
- CNN-urile utilizează frecvent kerneluri de dimensiuni **impare** (ex: 1, 3, 5 sau 7).
    - Kernelurile impare permit păstrarea dimensionalității spațiale prin utilizarea aceluiași număr de rânduri de *padding* sus și jos, și același număr de coloane stânga și dreapta.

Stride (Pasul)

- În calculul cross-correlation, se începe din colțul superior-stâng și se glisează fereastra, de obicei, **un singur element la un moment dat** (stride 1).
- **Stride** se referă la numărul de rânduri și coloane traversate per glisare. Notat ca s_h și s_w .
- **Motivație:** Un *stride* mai mare de 1 este folosit fie pentru **eficiență computațională**, fie pentru a efectua **downsampling** (reducerea drastică a dimensionalității).

Formula Generală a Dimensiunii Ieșirii

- În general, când padding-ul total este p_h rânduri și p_w coloane, și stride-ul este s_h și s_w, dimensiunea ieșirii este:

$$\lfloor (n_h - k_h + p_h + s_h)/s_h \rfloor \times \lfloor (n_w - k_w + p_w + s_w)/s_w \rfloor$$

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Zero Padding

- **Analogie Simplificată (Rama de Tablou):** Imaginea originală este ca o pictură valoroasă. Kernelul (lupa) se aplică pe ea. Fără padding, kernelul nu poate ajunge la marginile picturii. Padding-ul este ca adăugarea unei rame groase, neutre (pixeli de zero) în jurul picturii, permițând kernelului să scaneze chiar și colțurile originale, menținând informațiile de margine.

Concept Dificil: Rolul Kernelurilor Impare (1, 3, 5)

- **Context:** De ce preferăm kernelurile 3×3 sau 5×5?
- *Asta se întâmplă deoarece:* Kernelurile impare permit o **simetrie perfectă** a padding-ului. Dacă avem un kernel 3×3 ($k_h = 3$), avem nevoie de un padding $p = 2$ pentru a menține dimensiunea. Putem plasa exact 1 rând de zero deasupra și 1 rând de zero dedesubt (total 2), și la fel pe laterale. Dacă $k = 4$ (par), trebuie să avem un rând în plus pe o parte, făcând designul rețelei mai complex.

Stride și Downsampling

- **Context:** Deși *pooling* este principala metodă de *downsampling* (vom vedea mai târziu), stride-ul îndeplinește aceeași funcție.
- *Asta se întâmplă deoarece:* Un stride de $s = 2$ (sărind peste un element la fiecare pas) calculează doar jumătate din numărul de ieșiri pe fiecare axă, reducând dimensiunea imaginii la jumătate. Acest lucru este eficient, deoarece calculele sunt efectuate mai rar.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Adâncimea Rețelei):**
  - **CAUZĂ:** Dorința de a construi rețele foarte adânci (cu multe straturi).
  - **EFECT:** Pierderea cumulativă de pixeli devine inacceptabilă. Acest lucru impune utilizarea **padding-ului** (adesea pentru a menține dimensiunea constantă), permițând rețelelor să devină mai profunde fără ca imaginea să dispară.
- **Conexiunea cu Eficiența Computațională:** Folosirea unui *stride* mai mare de 1 este o modalitate directă de a crește eficiența, deoarece reduce numărul total de ieșiri calculate de stratul convoluțional.

---

5. EXERCITII (Calcul pe Straturi Convoluționale cu Padding și Stride)

Voi genera exerciții de calcul bazate pe formula generală a dimensiunii ieșirii.

Exercițiul P.4: Calculul Dimensiunii Ieșirii (Stridare și Padding)

Considerăm o operație de convoluție cu următorii parametri:

- Intrare: $n_h = 13$, $n_w = 13$
- Kernel: $k_h = 3$, $k_w = 3$
- Padding (total): $p_h = 2$, $p_w = 2$
- Stride: $s_h = 2$, $s_w = 2$

Calculați dimensiunea spațială a ieșirii ($n'_h \times n'_w$).

Formula necesară (Conform surselor, Ecuația 19):

$$n'_h = \lfloor (n_h - k_h + p_h + s_h)/s_h \rfloor$$
$$n'_w = \lfloor (n_w - k_w + p_w + s_w)/s_w \rfloor$$

Rezolvare P.4:

1. **Calculul Înălțimii Ieșirii (n'_h):**

$$n'_h = \lfloor (13 - 3 + 2 + 2)/2 \rfloor = \lfloor (14)/2 \rfloor = 7$$

2. **Calculul Lățimii Ieșirii (n'_w):**

$$n'_w = \lfloor (13 - 3 + 2 + 2)/2 \rfloor = \lfloor (14)/2 \rfloor = 7$$

Dimensiunea ieșirii este $\mathbf{7} \times \mathbf{7}$.

Exercițiul P.5: Determinarea Padding-ului Necesare pentru Conservarea Dimensiunii

Avem o intrare de $n_h = 28$ și $n_w = 28$. Dorim să folosim un kernel $k_h = 5, k_w = 5$ și un stride implicit $s = 1$.

1. Calculați dimensiunea ieșirii dacă se folosește *padding* zero (implicit, $p = 0$).
2. Calculați valoarea *totală* a padding-ului p_h și p_w necesar pentru a obține o ieșire cu aceeași dimensiune ca intrarea (28×28).

Rezolvare P.5:

1. **Dimensiunea Ieșirii (Padding Zero):**

$$n'_h = (28 - 5 + 1) = 24$$
$$n'_w = (28 - 5 + 1) = 24$$

   Dimensiunea ieșirii este $\mathbf{24} \times \mathbf{24}$. (S-a pierdut informația de margine).
2. **Padding-ul Total Necesare ($p = k - 1$, conform surselor):** Pentru a menține dimensiunea ($n'_h = n_h$), trebuie să avem: $n_h - k_h + p_h + 1 = n_h$. Simplificând, $p_h = k_h - 1$.

$$p_h = 5 - 1 = 4$$

$$p_w = 5 - 1 = 4$$

Este necesar un *padding total* de 4 rânduri și 4 coloane. (Acest lucru înseamnă 2 rânduri sus/jos și 2 coloane stânga/dreapta).

Exercițiul P.6: Cross-Correlation cu Stridare (Calcul Punctual)

Considerăm o operație de cross-correlation cu:

- Intrare (X): 5×5 (cu padding zero pe margine)
- Kernel (V): 3×3
- Stride: $s_h = 2$, $s_w = 2$
- Bias: $u = 0$

Calculați elementul de ieșire $\mathbf{H}_{1,1}$ (rândul 1, coloana 1, assuming indexare de la 0), având în vedere că fereastra se deplasează:

- Prima activare $\mathbf{H}_{0,0}$ începe la $(0, 0)$.
- A doua activare $\mathbf{H}_{1,0}$ începe la $(0 + s_h, 0) = (2, 0)$.
- A treia activare $\mathbf{H}_{1,1}$ începe la $(2, 0 + s_w) = (2, 2)$.

Să presupunem că Kernelul V și porțiunea de intrare X centrată pe $(2, 2)$ sunt:

$$V = [1 \quad 1 \quad 1\,0 \quad 0 \quad 0\,1 \quad 1 \quad 1], \quad X_{\text{patc}(2,2)} = [4 \quad 5 \quad 6\,7 \quad 8 \quad 9\,1 \quad 2 \quad 3]$$

Calculați valoarea elementului de ieșire $\mathbf{H}_{1,1}$.

Rezolvare P.6:

Elementul $\mathbf{H}_{1,1}$ este rezultatul înmulțirii element-wise și sumării dintre Kernelul V și porțiunea 3×3 din X (centrată pe $(2, 2)$):

$$\mathbf{H}_{1,1} = (1 \cdot 4) + (1 \cdot 5) + (1 \cdot 6) + (0 \cdot 7) + (0 \cdot 8) + (0 \cdot 9) + (1 \cdot 1) + (1 \cdot 2) + (1 \cdot 3)$$

$$\mathbf{H}_{1,1} = 4 + 5 + 6 + 0 + 0 + 0 + 1 + 2 + 3$$

$$\mathbf{H}_{1,1} = \mathbf{21}$$

3.4

Bun venit la o lecție esențială care duce înțelegerea noastră despre convoluții la nivelul următor: **3.4 Canale de Intrare Multiple și Canale de Ieșire Multiple**. Dacă până acum am simplificat majoritatea exemplelor la o singură intrare și o singură ieșire, realitatea imaginilor (precum cele color RGB) și a rețelelor profunde impune manipularea mai multor canale.

Ghid de Studiu Complet: 3.4 Canale de Intrare Multiple și Canale de Ieșire Multiple

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.4 detaliază modul în care straturile convoluționale gestionează complexitatea datelor din lumea reală, care sunt, de obicei, **tenzori tridimensionali**. Imaginile color, de exemplu, au canale standard **RGB** (Red, Green, Blue). Când adăugăm canale în ecuație, atât intrările, cât și reprezentările ascunse (ieșirile) devin **tenzori tridimensionali** (Canale $\times$ Înălțime $\times$ Lățime).

Mecanismul cheie este că operația de **cross-correlation** trebuie să se desfășoare pe *toate* canalele de intrare (C_{in}) simultan. Kernelul de convoluție trebuie să aibă același număr de canale de intrare ca și datele de intrare (C_{in}). Apoi, modelul folosește multiple kerneluri (câte unul pentru fiecare canal de ieșire dorit, C_{out}) pentru a produce *feature maps* separate.

Creșterea Adâncimii Canalelor: În arhitecturile neuronale populare, dimensiunea canalelor este adesea **crescută** pe măsură ce avansăm în rețea (către straturile superioare), făcând *downsampling* (reducerea rezoluției spațiale) pentru a face loc unei **adâncimi mai mari a canalelor** (*channel depth*).

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Dimensiunea Canale:** Intrările și ieșirile cu canale multiple devin tenzori tridimensionali. Axa de canale este numită **dimensiunea canalelor** (*channel dimension*).
2. **Mecanismul Kernelului de Intrare:** Dacă datele de intrare au C_{in} canale, kernelul de convoluție trebuie să aibă și el C_{in} canale. Cross-correlation este efectuată pe fiecare canal, iar rezultatele C_{in} sunt **însumate** (summing over the channels) pentru a produce un **tensor bidimensional** (o singură hartă de caracteristici).
3. **Mecanismul Kernelului de Ieșire:** Pentru a obține C_{out} canale de ieșire, se creează un kernel tensor de formă $C_{in} \times k_h \times k_w$ pentru *fiecare* canal de ieșire. Astfel, forma finală a kernelului convoluțional este $C_{out} \times C_{in} \times k_h \times k_w$.
4. **Convoluția 1×1:** Deși pare inutilă (nu corelează pixeli adiacenți), convoluția 1×1 este o operație populară utilizată pentru a efectua calculul **doar pe dimensiunea canalelor**. Aceasta acționează ca un strat complet conectat aplicat la fiecare poziție de pixel, transformând C_{in} valori de intrare în C_{out} valori de ieșire.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.4 Canale de Intrare Multiple și Canale de Ieșire Multiple

Trecerea la Tenzori 3D

- Până acum, exemplele numerice au simplificat intrarea la un singur canal de intrare și unul de ieșire.
- Când se adaugă canale, intrările și reprezentările ascunse devin **tenzori tridimensionali**.

- O imagine de intrare RGB are forma $3 \times h \times w$ (3 canale, înălțime, lățime). Axa cu dimensiunea 3 este **dimensiunea canalului**.

Kerneluri cu Canale de Intrare Multiple

- Când datele de intrare conțin canale multiple (C_{in}), kernelul de convoluție trebuie construit cu **același număr de canale de intrare** pentru a putea efectua cross-correlation.
- Dacă $C_{in} > 1$, kernelul trebuie să conțină un tensor de formă $k_h \times k_w$ pentru **fiecare canal de intrare**.
- Concatenarea acestor C_{in} tensori rezultă într-un kernel de convoluție de formă $C_{in} \times k_h \times k_w$.

Mecanismul de Calcul (Cross-Correlation pe Canale Multiple)

- Se efectuează o operație de cross-correlation pe tensorul bidimensional al intrării și pe tensorul bidimensional al kernelului **pentru fiecare canal**.
- Rezultatele din cele C_{in} canale sunt apoi **însumate** (adunate împreună) pentru a obține un **tensor bidimensional** (o singură hartă de caracteristici de ieșire).
- Exemplu (Figura 6): Pentru două canale de intrare, se calculează rezultatul parțial al Canalului 1 și al Canalului 2, iar cele două rezultate sunt adunate pentru a produce valoarea finală de ieșire.

Kerneluri cu Canale de Ieșire Multiple

- În arhitecturile neuronale, este esențial să avem canale de ieșire multiple (C_{out}) la fiecare strat.
- Pentru a obține o ieșire cu canale multiple, trebuie să creăm un kernel tensor de formă $C_{in} \times k_h \times k_w$ pentru **fiecare canal de ieșire**.
- Acestea sunt concatenate de-a lungul dimensiunii canalului de ieșire.
- Forma finală a kernelului de convoluție este $_{\mathbf{ou}} \times \ \times {}_{\mathbf{h}} \times {}_{\mathbf{w}}$.
- Rezultatul pe fiecare canal de ieșire este calculat din kernelul corespunzător acelui canal de ieșire și ia în considerare **toate canalele din tensorul de intrare**.

Rolul Intuitiv al Canalelor de Ieșire

- Intuitiv, am putea crede că fiecare canal de ieșire răspunde unui **set diferit de caracteristici**.
- Totuși, reprezentările nu sunt învățate independent, ci sunt optimizate pentru a fi **utilizate în comun (*jointly useful*)**.
- Nu este neapărat ca un singur canal să învețe un detector de margini, ci mai degrabă o anumită direcție în spațiul canalului să corespundă detectării marginilor.

Convoluția 1×1

- La prima vedere, o convoluție 1×1 (unde $k_h = k_w = 1$) nu pare să aibă sens, deoarece convoluția corelează pixeli adiacenți, iar un kernel 1×1 nu o face.
- Totuși, aceste operații sunt populare în designul rețelelor profunde complexe.
- Convoluția 1×1 **pierde capacitatea de a recunoaște tipare** care constau din interacțiuni între elementele adiacente în dimensiunile de înălțime și lățime.
- Calculul convoluției 1×1 are loc **doar pe dimensiunea canalului**.
- Putem considera stratul convoluțional 1×1 ca fiind un **strat complet conectat** aplicat la fiecare locație de pixel, transformând valorile de intrare C_{in} în valorile de ieșire C_{out}.
- Deoarece este tot un strat convoluțional, ponderile sunt **legate (partajate) pe toate locațiile de pixel**.
- Un strat convoluțional 1×1 necesită $C_{out} \times C_{in}$ greutăți (plus bias-ul).

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Sumarea Peste Canalele de Intrare

- **Analogie (Fuziunea Culorilor):** Imaginați-vă că procesați o imagine RGB. Kernelul are o componentă R, o componentă G și o componentă B. Fiecare componentă a kernelului se înmulțește cu canalul său corespunzător. După înmulțirea element-wise, suma finală (un singur număr) nu se mai face doar spațial (pe $k_h \times k_w$), ci **spațial ȘI pe canale**.
- *Asta se întâmplă deoarece:* Această însumare (summing over C_{in}) fuzionează informația din toate canalele de intrare într-o singură valoare, permițând, de exemplu, unui singur *feature map* de ieșire să detecteze marginea roșie **și** verde simultan.

Concept Dificil: Convoluția 1×1 ca Strat Complet Conectat

- **Analogie Simplificată (Ajustarea Tonurilor):** Imaginați-vă un filtru pe telefon. Un 1×1 convoluție nu se uită la vecini (adică nu este interesat de modelul spațial local). În schimb, la fiecare pixel, el primește trei valori (R, G, B) și le combină liniar pentru a produce două noi valori (de exemplu, un canal de luminozitate și un canal de contrast). Acesta este exact ceea ce face un strat complet conectat!
- *Asta se întâmplă deoarece:* Deoarece ponderile sunt partajate pe toate locațiile spațiale, este ca și cum același strat complet conectat ar fi aplicat **independent** la fiecare pixel.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Adâncimea Canalului):**
  - **CAUZĂ:** Dorința de a obține o reprezentare semantică mai bogată în straturile superioare.

- **EFECT:** Rețelele populare **cresc dimensiunea canalului** (C_{out} mai mare decât C_{in}) pe măsură ce se avansează în rețea, în timp ce adesea se **pierde rezoluția spațială** (*downsampling* se face pentru a compensa).
- **Conexiunea cu Secțiunea 3.1 (Eficiența Parametrilor):** Introducerea lui C_{in} și C_{out} mărește numărul total de parametri. De exemplu, un kernel 5×5 cu 3 intrări și 64 de ieșiri are $64 \times 3 \times 25 = 4800$ de parametri, ceea ce este încă infinit mai mic decât un MLP, dar semnificativ mai mare decât un kernel 5×5 cu 1×1 canale.

---

5. EXERCITII (Calcul pe Straturi Convoluționale cu Canale Multiple)

Vom genera exerciții bazate pe calculul numărului de parametri și pe operația de cross-correlation cu canale multiple.

Exercițiul M.4: Calculul Numărului Total de Parametri (Canale Multiple)

Definiți un strat convoluțional cu următorii parametri:

- Canale de intrare $C_{in} = 3$ (RGB).
- Canale de ieșire $C_{out} = 64$.
- Dimensiunea kernel-ului $k_h \times k_w = 7 \times 7$.

1. Calculați numărul total de greutăți V (kernelii).
2. Calculați numărul total de bias-uri u.
3. Calculați numărul total de parametri reali.

Rezolvare M.4:

1. **Numărul de Greutăți (V):**

$$V = C_{out} \times C_{in} \times k_h \times k_w$$
$$V = 64 \times 3 \times 7 \times 7 = 64 \times 3 \times 49 = 192 \times 49 = \mathbf{9408}$$

2. **Numărul de Bias-uri (u):** Bias-ul este aplicat un singur scalar pentru fiecare canal de ieșire.

$$u = C_{out} = \mathbf{64}$$

3. **Total Parametri:**

$$\text{Total parametri} = 9408 + 64 = \mathbf{9472}$$

Exercițiul M.5: Cross-Correlation cu Canale Multiple (Calcul Punctual)

Considerăm o operație de cross-correlation pentru a calcula un singur element de ieșire (de exemplu, $H_{0,0,1}$), cu $C_{in} = 2$ și $k_h \times k_w = 2 \times 2$. Bias-ul canalului de ieșire 1 este $u_1 = 3$.

Intrări (patch 2×2):

$$\text{analul Intrare 1: } X_{c=1} = [1 \quad 2\ 3 \quad 4]$$

$$\text{analul Intrare 2: } X_{c=2} = [5 \quad 6\ 7 \quad 8]$$

Kernel (pentru Canalul de Ieșire 1):

$$\text{ernel Intrare 1: } V_{,,c=1,d=1} = [1 \quad 1\ 0 \quad 0]$$

$$\text{ernel Intrare 2: } V_{,,c=2,d=1} = [0 \quad 1\ 1 \quad 0]$$

Calculați valoarea elementului de ieșire $\mathbf{H}_{0,0,1}$.

Formula de calcul (conform):

$$[\mathbf{H}]_{i,j,d} = [\mathbf{u}]_d + \sum_{c=1}^{C_{in}} \left(\sum_{a,b} [\mathbf{V}]_{a,b,c,d} [\mathbf{X}]_{i+a,j+b,c} \right)$$

Rezolvare M.5:

Pasul 1: Contribuția din Canalul de Intrare 1 ($c = 1$):

$$\text{Suma}_1 = (1 \cdot 1) + (1 \cdot 2) + (0 \cdot 3) + (0 \cdot 4) = 1 + 2 + 0 + 0 = 3$$

Pasul 2: Contribuția din Canalul de Intrare 2 ($c = 2$):

$$\text{Suma}_2 = (0 \cdot 5) + (1 \cdot 6) + (1 \cdot 7) + (0 \cdot 8) = 0 + 6 + 7 + 0 = 13$$

Pasul 3: Adăugarea Bias-ului și Sumarea Contribuțiilor (Fusionarea Canalelor): Bias-ul $[\mathbf{u}]_1 = 3$.

$$[\mathbf{H}]_{0,0,1} = [\mathbf{u}]_1 + \text{Suma}_1 + \text{Suma}_2$$

$$[\mathbf{H}]_{0,0,1} = 3 + 3 + 13 = \mathbf{19}$$

Exercițiul M.6: Convoluția 1×1

Considerați un strat de convoluție 1×1 cu $C_{in} = 4$ și $C_{out} = 1$ (un singur *feature map* de ieșire).

1. Câte greutăți (W) sunt necesare pentru acest strat?
2. Dacă la o anumită poziție de pixel, vectorul de intrare pe canale este $X_{\text{pixel}} = [10\ 5\ 1\ 0]$, iar ponderile kernelului 1×1 sunt $V = [0.5 \quad 0.2 \quad -0.1 \quad 1.0]^\top$ și bias-ul este $u = 1$. Care este valoarea de ieșire a acestui pixel?

Rezolvare M.6:

1. **Număr de Greutăți:** Un strat 1×1 necesită $C_{out} \times C_{in}$ greutăți.

$$\text{Număr greutăți} = 1 \times 4 = \mathbf{4}$$

2. **Valoarea de ieșire:** Această operație este o combinație liniară (produs scalar) între vectorul de intrare pe canale și kernel:

$$\mathbf{H} = u + V^{\top} X_{\text{pixel}}$$

$$\mathbf{H} = 1 + (0.5 \cdot 10) + (0.2 \cdot 5) + (-0.1 \cdot 1) + (1.0 \cdot 0)$$

$$\mathbf{H} = 1 + 5 + 1 - 0.1 + 0 = \mathbf{6.9}$$

3.5

Acest capitol, **3.5 Pooling (Agregarea)**, este vital deoarece introduce un alt element structural esențial al CNN-urilor, destinat să rezolve problemele de stabilitate la locație și de scalare a imaginii. *Pooling*-ul ne permite să trecem de la detectarea caracteristicilor locale la o înțelegere globală a imaginii, păstrând în același timp avantajele straturilor convoluționale.

Ghid de Studiu Complet: 3.5 Pooling (Agregarea)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.5 introduce straturile de **Pooling** (Agregare), care îndeplinesc două scopuri esențiale în arhitecturile CNN: **reducerea graduală a rezoluției spațiale** (*downsampling*) și **mitigarea sensibilității la locație** (obținerea invarianței la translație). Pe măsură ce avansăm în rețea, dorim să agregăm informațiile pentru a crește **câmpul receptiv** al fiecărui nod ascuns, astfel încât straturile finale să fie sensibile la întregul input (o reprezentare globală). Spre deosebire de straturile convoluționale, straturile de *pooling* sunt **deterministe** și **nu conțin parametri** (nu au *kernel* care trebuie învățat). Acestea funcționează aplicând o fereastră fixă (pooling window) care glisează peste intrare, calculând fie valoarea **maximă** (***max-pooling***), fie **valoarea medie** (***average pooling***) dintr-o regiune.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Scopul Dual:** *Pooling*-ul servește la reducerea spațială a reprezentărilor și la atenuarea sensibilității straturilor convoluționale la locație.
2. **Invarianța la Translație:** *Pooling*-ul ajută la obținerea unei **invarianțe la translație** ușoare, asigurându-se că detectarea unei caracteristici (ex: o margine) rămâne stabilă chiar dacă acea caracteristică se deplasează cu un pixel.
3. **Fără Parametri:** Straturile de *pooling* sunt **deterministe** și **nu conțin parametri** (nu au kernel).
4. **Tipuri Principale:** Operațiile de *pooling* folosesc fie **maximum pooling** (valoarea maximă), fie **average pooling** (valoarea medie) a elementelor din fereastra de *pooling*.
5. **Manipularea Canalelor:** Straturile de *pooling* aplică agregarea pe **fiecare canal de intrare separat**; ele nu însumează intrările peste canale (spre deosebire de convoluție),

astfel încât **numărul de canale de ieșire este același cu numărul de canale de intrare**.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.5 Pooling (Agregarea)

Motivație și Obiective

- La prelucrarea imaginilor, se dorește reducerea treptată a rezoluției spațiale a reprezentărilor ascunse.
- Scopul este agregarea informației pentru a mări **câmpul receptiv** (în intrare) la care este sensibil fiecare nod ascuns.
- Sarcinile finale (ex: "imaginea conține o pisică?") necesită o întrebare globală despre întreaga imagine, iar unitățile stratului final ar trebui să fie sensibile la întregul input.
- Agregarea informației produce hărți din ce în ce mai grosiere (*coarser maps*), atingând scopul de a învăța o reprezentare globală.

Necesitatea Invarianței la Translație

- La detectarea caracteristicilor de nivel inferior (precum muchii), se dorește ca reprezentările să fie **oarecum invariante la translație**.
- În realitate, obiectele nu apar niciodată exact în același loc.
- O mică vibrație a camerei sau o translație de un pixel poate schimba vast ieșirea imaginii față de imaginea originală $\mathbf{X}$.

Funcția Straturilor de Pooling

- Secțiunea introduce straturile de *pooling*, care servesc scopului dual de a **mitiga sensibilitatea straturilor convoluționale la locație** și de a efectua **downsampling spațial** al reprezentărilor.
- Operatorii de *pooling* constau dintr-o **fereastră de formă fixă** (*pooling window*) care glisează peste toate regiunile din intrare, conform *stride*-ului său.
- La fiecare locație, operatorul calculează o singură ieșire.
- **Diferența cheie:** Spre deosebire de straturile convoluționale, stratul de *pooling* **nu conține parametri** (nu există kernel).

Tipuri de Operații de Pooling

- Operatorii de *pooling* sunt **deterministi**.
- Tipic, ei calculează fie **maximum** fie **average** valoarea elementelor din fereastra de *pooling*.
    - **Maximum pooling (max-pooling)**: Calculează maximul.

    - **Average pooling**: Calculează media.
- Fereastra de *pooling* începe din colțul superior-stâng și glisează de la stânga la dreapta și de sus în jos.
- O operație de *pooling* $p \times q$ folosește o fereastră $p \times q$.

Exemplu de Max-Pooling (Figura 8)

- Pentru un 2×2 Max Pooling, ieșirea 2×2 este derivată din valoarea maximă din fiecare fereastră.
    - $\max(0, 1, 3, 4) = 4$.
    - $\max(1, 2, 4, 5) = 5$.
    - $\max(3, 4, 6, 7) = 7$.
    - $\max(4, 5, 7, 8) = 8$.

Achiziționarea Invarianței la Translație (Exemplul Marginii)

- Folosind un strat de *maximum pooling* 2×2, se poate detecta în continuare un tipar recunoscut de stratul convoluțional, chiar dacă acel tipar se deplasează cu **nu mai mult de un element în înălțime sau lățime**.
- Stratul de *pooling* va produce o ieșire constantă $Y_{i,j} = 1$ (în exemplul menționat) chiar dacă valorile inputului s-au schimbat ușor din cauza translației.

Controlul Formei și Canalele

- La fel ca straturile convoluționale, straturile de *pooling* pot schimba forma ieșirii prin aplicarea de **padding** și ajustarea **stride**-ului.
- Când se prelucrează date de intrare cu canale multiple, stratul de *pooling* efectuează agregarea **separat pe fiecare canal de intrare**.
- Acest lucru diferă de stratul convoluțional, care însumează intrările peste canale.
- Drept urmare, **numărul de canale de ieșire al stratului de *pooling* este același cu numărul de canale de intrare**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Deterministic și Fără Parametri

- **Explicație Simplificată (ELI5):** Stratul convoluțional este ca un student care trebuie să învețe din date (parametrii V și u se schimbă). Stratul de *pooling* este ca un calculator. Calculatorul știe o regulă fixă (fie "găsește cel mai mare număr," fie "calculează media") și aplică acea regulă indiferent de date. Deoarece nu există nimic de învățat sau de actualizat prin *backpropagation* (nu există kernel), se spune că nu are parametri.

Concept Dificil: Invarianța la Translație prin Max Pooling

- **Analogie (Ochii unei Pisici):** Să presupunem că un filtru convoluțional detectează un ochi de pisică (output mare). Dacă pisica se mișcă puțin (translație de un pixel), filtrul s-ar putea să rateze ochiul în acea poziție. *Max-pooling* rezolvă asta: dacă un filtru detectează ochiul într-o regiune 2×2, chiar dacă ochiul se mută la pixelul vecin, valoarea maximă din fereastra 2×2 (care include acum noua locație) **rămâne mare**. Astfel, stratul de ieșire agregat știe că "un ochi există în această regiune generală", indiferent de poziția exactă.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu Câmpul Receptiv (Lecția 3.2):** *Pooling*-ul este instrumentul principal prin care se mărește progresiv câmpul receptiv. Prin reducerea rezoluției spațiale (downsampling), nodurile din straturile superioare pot "vedea" o porțiune mai mare a imaginii inițiale.
- **Diferența față de Convoluție (Lecția 3.2, 3.4):** Ambele folosesc o fereastră glisantă (*sliding window*) și *stride*/ *padding* pentru a controla forma ieșirii. Însă, stratul convoluțional **însumează** informația pe canalele de intrare (fuziune), în timp ce stratul de *pooling* lucrează **independent pe fiecare canal**.

---

5. EXERCITII (Calcul pe Straturi de Pooling)

Voi genera exerciții de calcul bazate pe operațiile de *max-pooling* și *average-pooling* și pe gestionarea dimensiunilor.

Exercițiul P.7: Calculul Max Pooling 2D

Considerați un strat de *Max Pooling* cu o fereastră 2×2 și un *stride* de 2.

- Intrare (X): 4×4

$$X = [1 \quad 5 \quad 2 \quad 8\,0 \quad 9 \quad 3 \quad 4\,7 \quad 6 \quad 10 \quad 1\,2 \quad 1 \quad 0 \quad 3]$$

Calculați tensorul de ieșire $\mathbf{Y}$.

Rezolvare P.7:

Deoarece *stride*-ul este 2, ferestrele de 2×2 nu se suprapun. Vom calcula 4 elemente de ieșire (2×2):

1. **Elementul $\mathbf{Y}_{0,0}$ (Fereastra $X_{0:2,0:2}$):**

$$\max(1, 5, 0, 9) = \mathbf{9}$$

2. **Elementul $\mathbf{Y}_{0,1}$ (Fereastra $X_{0:2,2:4}$):**

$$\max(2, 8, 3, 4) = \mathbf{8}$$

3. **Elementul $\mathbf{Y}_{1,0}$ (Fereastra $X_{2:4,0:2}$):**

$$\max(7, 6, 2, 1) = \mathbf{7}$$

4. **Elementul $\mathbf{Y}_{1,1}$ (Fereastra $X_{2:4,2:4}$):**

$$\max(10, 1, 0, 3) = \mathbf{10}$$

Tensorul de ieșire $\mathbf{Y}$ este:

$$\mathbf{Y} = [9 \quad 8\ 7 \quad 10]$$

Exercițiul P.8: Calculul Average Pooling (Media)

Considerați un strat de *Average Pooling* cu o fereastră 3×3 și un *stride* de 1.

- Intrare (X): 3×3

$$X = [1 \quad 2 \quad 3\ 4 \quad 5 \quad 6\ 7 \quad 8 \quad 9]$$

Calculați tensorul de ieșire $\mathbf{Y}$.

Rezolvare P.8:

Deoarece intrarea și fereastra au aceeași dimensiune (3×3), ieșirea va fi un singur element 1×1.

- **Elementul $\mathbf{Y}_{0,0}$:** Se calculează media tuturor elementelor din X:

$$\text{Suma} = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$$
$$\text{Media} = 45/9 = \mathbf{5}$$

Tensorul de ieșire $\mathbf{Y}$ este:

$$\mathbf{Y} = [5]$$

Exercițiul P.9: Calculul Dimensiunii Ieșirii și Canalele

Definiți un strat de *Pooling* cu următorii parametri:

- Intrare: $n_h = 10$, $n_w = 10$.
- Număr de canale de intrare $C_{in} = 32$.
- Fereastră de *pooling*: $k_h = 3$, $k_w = 3$.
- Stride: $s_h = 2$, $s_w = 2$.
- Padding: $p_h = 0$, $p_w = 0$.

1. Calculați dimensiunea spațială a ieșirii ($n'_h \times n'_w$).
2. Care este numărul de canale de ieșire C_{out}?

Formula necesară (Conform Lecției 3.3, valabilă și pentru pooling):

$$n'_h = \lfloor (n_h - k_h + p_h)/s_h \rfloor + 1$$

(Am ajustat formula din sursă 3.3 pentru a corespunde notației standard $\lfloor (n_h - k_h + p_h)/s_h \rfloor + 1$, care este echivalentă cu formula din sursă doar dacă p_h și s_h sunt corelate corespunzător.) Vom folosi formula standard de calcul pentru a fi preciși.

Rezolvare P.9:

1. **Calculul Dimensiunii Spațiale a Ieșirii:**
   - Înălțimea n'_h:

     $$n'_h = \lfloor (10 - 3 + 0)/2 \rfloor + 1 = \lfloor 7/2 \rfloor + 1 = 3 + 1 = 4$$

   - Lățimea n'_w:

     $$n'_w = \lfloor (10 - 3 + 0)/2 \rfloor + 1 = 4$$

     Dimensiunea spațială a ieșirii este $\mathbf{4 \times 4}$.
2. **Numărul de Canale de Ieșire (C_{out}):** Stratul de *pooling* lucrează separat pe fiecare canal de intrare, fără a le însuma.

   $$C_{out} = C_{in} = \mathbf{32}$$

3.6

Lecția de astăzi este una de referință. **3.6 Rețele Neuronale Convoluționale (LeNet)** marchează momentul în care punem cap la cap toate componentele teoretice (convoluție, padding, stride, pooling) pentru a înțelege și analiza prima arhitectură CNN de succes publicată.

Ghid de Studiu Complet: 3.6 Rețele Neuronale Convoluționale (LeNet)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.6 prezintă **LeNet** (cunoscut și ca **LeNet-5**), o arhitectură pionieră care a demonstrat pentru prima dată, cu succes, potențialul Rețelelor Neuronale Convoluționale (CNNs) pentru sarcini de viziune computerizată. Introdus de **Yann LeCun în 1989**, acest model a reprezentat o culminare a cercetării de un deceniu. Scopul său principal a fost **recunoașterea cifrelor scrise de mână în imagini**, fiind ulterior adaptat pentru a procesa

depozite în **ATM-uri** în anii 1990. LeNet a atins rezultate remarcabile, egalând performanța *support vector machines* (SVM), o abordare dominantă la acea vreme.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Arhitectură Duală:** LeNet-5 este format din două blocuri principale: un **codificator convoluțional** (două straturi convoluționale urmate de pooling) și un **bloc dens** (trei straturi complet conectate).
2. **Pionieratul CNN:** Yann LeCun a publicat primul studiu care a antrenat cu succes CNN-uri prin **backpropagation în 1989**.
3. **Componente Standardizate (Istorice):** Fiecare bloc convoluțional folosea un strat convoluțional, o funcție de activare **Sigmoid** și o operație de **Average Pooling**. (Deși astăzi se preferă ReLU și Max-Pooling, acestea nu fuseseră descoperite la acea vreme).
4. **Tranziția Crucială:** Pentru a trece de la blocul convoluțional la blocul dens, ieșirea multi-dimensională (Channels, Height, Width) trebuie **aplatizată (*flatten*)** într-un vector unidimensional.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.6 Rețele Neuronale Convoluționale (LeNet)

- **Istoric:** Modelul a fost introdus de Yann LeCun, pe când era cercetător la AT&T Bell Labs.
- **Scop:** Recunoașterea cifrelor scrise de mână în imagini.
- **Impact:** A atins rezultate remarcabile, egalând performanța mașinilor cu vectori suport (SVM). Codul lui LeCun și al colegului său, Leon Bottou, este rulat și astăzi de unele ATM-uri pentru procesarea depozitelor.

Structura la Nivel Înalt (LeNet-5)

LeNet-5 este compus din două părți majore:

1. Un **codificator convoluțional** format din **două straturi convoluționale**.
2. Un **bloc dens** format din **trei straturi complet conectate (Fully-Connected - FC)**.

Fluxul Datelor și Detalii Arhitecturale (Conform Figurii 10)

- **Intrare:** O imagine cu un singur canal (alb-negru) de dimensiunea 28×28.
- **C1 (Primul Strat Convoluțional):**
  - Folosește un kernel de 5×5.
  - Are **6 canale de ieșire**.
  - Utilizează **2 pixeli de padding** (pe înălțime și lățime) pentru a compensa reducerea dimensionalității cauzată de kernelul 5×5.

    - Ieșirea păstrează dimensiunea spațială: $6@28 \times 28$.
- **S2 (Primul Strat de Pooling):**
    - Folosește o operație de 2×2 **Average Pooling** cu **stride 2**.
    - Stratul de *pooling* **înjumătățește înălțimea și lățimea** reprezentării.
    - Ieșirea este $6@14 \times 14$.
- **C3 (Al Doilea Strat Convoluțional):**
    - Folosește un kernel de 5×5.
    - Are **16 canale de ieșire**.
    - **Nu folosește padding**.
    - Ieșirea este redusă spațial la $16@10 \times 10$.
- **S4 (Al Doilea Strat de Pooling):**
    - Folosește o operație de 2×2 **Average Pooling** cu **stride 2**.
    - Ieșirea este redusă din nou la jumătate: $16@5 \times 5$.
- **Tranziția la Blocul Dens:**
    - Ieșirea S4 (care este un tensor 4D în timpul mini-batch-ului) este **aplatizată (*flatten*)** pentru a fi transmisă straturilor complet conectate (FC).
    - Dimensiunea aplatizată de intrare în FC1 este $16 \times 5 \times 5 = 400$.
- **Blocul Dens (FC):** Trei straturi complet conectate reduc progresiv dimensionalitatea:
    - FC1 (F5): 120 ieșiri.
    - FC2 (F6): 84 ieșiri.
    - FC3 (Output): 10 ieșiri (corespunzând celor 10 clase posibile de cifre).
- **Activări:** Fiecare strat convoluțional utilizează o funcție de activare **Sigmoid**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Aplatizarea (*Flattening*)

- **Explicație Simplificată (Transformarea):** Blocul convoluțional (C1, S2, C3, S4) extrage caracteristici spațiale (linii, curbe, colțuri). Odată ce aceste caracteristici sunt extrase și agregate spațial, nu mai avem nevoie de structura de grilă. Aplatizarea ia acești $16 \times 5 \times 5 = 400$ de neuroni și îi înșiră într-un singur vector lung de 400 de elemente.
- *Asta se întâmplă deoarece:* Straturile complet conectate (FC) sunt concepute să lucreze cu vectori (date tabulare), nu cu tenzori spațiali. Ele preiau vectorul de 400 de elemente și îl folosesc pentru a realiza clasificarea finală (reducerea la 10 scoruri de clasă).

Concept Dificil: Rolul Sigmoidului și Average Pooling (Context Istoric)

- *Asta se întâmplă deoarece:* În anii 1990, Sigmoid și Average Pooling erau alegerile naturale, moștenite din rețelele neuronale tradiționale. Deși astăzi știm că ReLU și Max-Pooling oferă performanțe mai bune (datorită stabilității numerice și a capacității de a detecta caracteristici mai puternice), LeNet a fost limitat de cunoștințele de la acea vreme.

Reducerea Dimensionalității prin Pooling

- O operație de pooling 2×2 cu stride 2 reduce dimensionalitatea spațială cu un **factor de 4** (jumătate pe înălțime, jumătate pe lățime, $1/2 \times 1/2 = 1/4$). Acest lucru crește eficiența și mărește câmpul receptiv.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Arhitectură):**
  - **CAUZĂ:** Necesitatea de a detecta tipare locale (caracteristici) pe o imagine.
  - **EFECT:** Utilizarea blocului convoluțional (C1/C3) cu partajarea ponderilor și conectivitate locală, urmată de *downsampling* prin pooling (S2/S4) pentru a obține invarianță locală și eficiență.
- **Conexiunea cu Secțiunea 3.1 & 3.3:** LeNet este exemplul practic al aplicării principiilor din Secțiunile 3.1 (înlocuirea FC cu Conv) și 3.3 (utilizarea padding-ului de 2 pixeli în C1 pentru a menține dimensiunea 28×28 în ciuda kernelului 5×5).

---

5. EXERCITII (Calcul pe Arhitectura LeNet-5)

Vom genera exerciții bazate pe fluxul de date și costul parametrilor din arhitectura LeNet, conform structurii din surse.

Exercițiul L.1: Analiza Fluxului Dimensional (Dimensionality Tracking)

Urmăriți dimensiunea spațială (Înălțime $\times$ Lățime) și numărul de canale (Adâncime) de-a lungul primului bloc convoluțional LeNet.

- Start: Imagine 28×28, 1 canal.
- Stratul C1: 5×5 Conv, 6 ieșiri, Padding 2.
- Stratul S2: 2×2 AvgPool, Stride 2.
- Stratul C3: 5×5 Conv, 16 ieșiri, Fără Padding.
- Stratul S4: 2×2 AvgPool, Stride 2.

Completați dimensiunea tensorului de ieșire (Canale @ Înălțime $\times$ Lățime) pentru fiecare strat.

Rezolvare L.1:

Strat	Formula dimensională (h/w)	Canale de ieșire	Dimensiunea ieșirii (H x W)
Intrare	N/A	1	28×28
C1	$(28 - 5 + 2 \times 2)/1 + 1 = 28$	6	$\mathbf{6@28 \times 28}$
S2	$28/2 = 14$	6	$\mathbf{6@14 \times 14}$
C3	$(14 - 5 + 0 \times 2)/1 + 1 = 10$	16	$\mathbf{16@10 \times 10}$
S4	$10/2 = 5$	16	$\mathbf{16@5 \times 5}$

Exercițiul L.2: Calculul Parametrilor (Blocul Convoluțional)

Calculați numărul total de parametri reali (Greutăți V și Bias-uri u) pentru cele două straturi convoluționale (C1 și C3) din LeNet-5.

- **Stratul C1:** $C_{in} = 1, C_{out} = 6, k = 5 \times 5$.
- **Stratul C3:** $C_{in} = 6, C_{out} = 16, k = 5 \times 5$.

Rezolvare L.2:

1. **Parametri C1:**
   - Greutăți V: $C_{out} \times C_{in} \times k_h \times k_w = 6 \times 1 \times 5 \times 5 = 150$.
   - Bias-uri u: $C_{out} = 6$.
   - Total C1: $150 + 6 = \mathbf{156}$.
2. **Parametri C3:**
   - Greutăți V: $C_{out} \times C_{in} \times k_h \times k_w = 16 \times 6 \times 5 \times 5 = 16 \times 150 = 2400$.
   - Bias-uri u: $C_{out} = 16$.
   - Total C3: $2400 + 16 = \mathbf{2416}$.
3. **Total Parametri Conv Block:** $156 + 2416 = \mathbf{2572}$.

Exercițiul L.3: Calculul Parametrilor (Blocul Dens)

Calculați numărul total de parametri reali pentru blocul dens (FC1, FC2, FC3) al rețelei LeNet.

- Intrare FC1 (F5): Ieșirea S4 aplatizată.
- FC1 $\rightarrow$ FC2: 120 ieșiri.
- FC2 $\rightarrow$ FC3: 84 ieșiri.
- FC3 $\rightarrow$ Ieșire: 10 ieșiri.

Rezolvare L.3:

1. **Intrare FC1:** Ieșirea S4 este $16@5 \times 5$. Număr intrări $N_{\text{in}} = 16 \times 5 \times 5 = 400$.
2. **Parametri FC1 (400 $\rightarrow$ 120):**
   - Ponderi W: $400 \times 120 = 48,000$.
   - Bias-uri b: 120.
   - Total FC1: $48,000 + 120 = 48,120$.
3. **Parametri FC2 (120 $\rightarrow$ 84):**
   - Ponderi W: $120 \times 84 = 10,080$.
   - Bias-uri b: 84.
   - Total FC2: $10,080 + 84 = 10,164$.
4. **Parametri FC3 (84 $\rightarrow$ 10):**
   - Ponderi W: $84 \times 10 = 840$.
   - Bias-uri b: 10.
   - Total FC3: $840 + 10 = 850$.
5. **Total Parametri Bloc Dens:** $48,120 + 10,164 + 850 = \mathbf{59,134}$.
6. **Total Parametri LeNet (Aproximativ):** $2,572 + 59,134 = \mathbf{61,706}$.

3.7

Bună ziua! Această lecție, **3.7 Rețele Neuronale Convoluționale Profunde (AlexNet)**, marchează o bornă istorică, reprezentând punctul de cotitură în care *deep learning*-ul a început să domine Viziunea Computerizată, punând bazele arhitecturilor moderne.

Ghid de Studiu Complet: 3.7 Rețele Neuronale Convoluționale Profunde (AlexNet)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.7 ne poartă printr-o retrospectivă istorică de la modelul LeNet la triumful **AlexNet** în competiția **ImageNet Challenge din 2012**. Până în acel moment, în ciuda existenței CNN-urilor (mulțumită LeNet), metodele tradiționale bazate pe *feature engineering* manual, cum ar fi **Support Vector Machines (SVMs)**, dominau. Descoperirea majoră din 2012, realizată de Alex Krizhevsky, Ilya Sutskever și Geoffrey Hinton, a fost atribuită nu doar unui design arhitectural mai adânc, ci și a doi factori cheie: **disponibilitatea datelor masive (ImageNet)** și **progresele hardware (GPU-uri)**. AlexNet a dovedit pentru prima dată că **trăsăturile învățate pot transcende trăsăturile proiectate manual**.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Revoluția din 2012:** AlexNet (o rețea CNN cu **8 straturi**) a câștigat competiția ImageNet Challenge 2012 cu o marjă fenomenal de mare, spulberând paradigma anterioară

bazată pe extragerea manuală de caracteristici.

2. **Factorii Cheie:** Succesul s-a datorat combinației dintre **setul de date ImageNet** (lansat în 2009, 1 milion de exemple din 1000 de categorii) și utilizarea **unităților de procesare grafică (GPU-uri)**, care sunt optimizate pentru operațiile de algebră liniară costisitoare din *deep learning*.
3. **Diferențe Arhitecturale (Față de LeNet):** AlexNet este **mult mai adânc** și a înlocuit funcția de activare **Sigmoid** cu **ReLU**. De asemenea, a folosit **Max Pooling** și a inclus tehnici moderne de regularizare precum **Dropout**.
4. **Reprezentări Ierarhice:** AlexNet a demonstrat că straturile inferioare învață descriptori care seamănă cu filtrele tradiționale (margini, culori), în timp ce straturile superioare construiesc structuri mai mari (ochi, nas) și, în cele din urmă, obiecte întregi.
5. **Costul Parametrilor:** Straturile complet conectate din AlexNet (cu 4096 de ieșiri) sunt masive, generând aproape **1 GB de parametri**.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.7 Rețele Neuronale Convoluționale Profunde (AlexNet)

Contextul Istoric (Până în 2012)

- Deși CNN-urile (după introducerea LeNet) erau cunoscute, ele nu au dominat imediat domeniul.
- În anii 1990 și începutul anilor 2000, rețelele neuronale erau adesea depășite de alte metode, precum **Support Vector Machines** (SVMs).
- Progresul în viziunea computerizată se baza pe **extracția manuală de caracteristici** (*manually engineering feature extraction pipelines*), unde caracteristicile erau "create" (*crafted*), nu învățate.
- Sistemele clasice erau procese în mai mulți pași (ex: preprocesare cu SIFT/SURF, urmată de un clasificator simplu).
- Modelele profunde nu erau fezabile din cauza:
  - Seturilor de date mici (adesea doar sute sau mii de imagini cu rezoluție scăzută, cum ar fi colecția UCI).
  - Hardware-ului insuficient de puternic pentru CNN-uri multi-strat cu mulți parametri.
  - Lipsa unor *trick*-uri cheie de antrenare (heuristici de inițializare, funcții de activare non-squashing, regularizare eficientă).

Factorul 1: Datele (ImageNet)

- Modelele profunde necesită **cantități mari de date** pentru a depăși metodele tradiționale.
- În **2009**, setul de date **ImageNet** a fost lansat, provocând cercetătorii să învețe modele din **1 milion de exemple**, fiecare aparținând uneia dintre cele **1000 de categorii**

distincte de obiecte.

Factorul 2: Hardware-ul (GPU-uri)

- Modelele de *deep learning* sunt consumatori masivi de cicluri de calcul.
- **Unitățile de Procesare Grafică (GPU-uri)** au schimbat jocul, fiind optimizate pentru operații de algebră liniară intensivă (precum produsele matriciale 4×4 necesare în grafica computerizată).
- Acest calcul este **similar** cu cel necesar pentru straturile convoluționale.
- În **2012**, Alex Krizhevsky și Ilya Sutskever au implementat un CNN profund care rula pe hardware GPU, folosind **două NVIDIA GTX 580s cu 3GB de memorie**.

Arhitectura AlexNet (2012)

- AlexNet a fost propus de Alex Krizhevsky, Ilya Sutskever și Geoffrey Hinton.
- Este o rețea CNN cu **8 straturi**: **cinci straturi convoluționale**, **două straturi ascunse complet conectate** și **un strat de ieșire complet conectat**.

Diferențe Cheie fața de LeNet

1. **Adâncime:** AlexNet este **mult mai adânc** decât LeNet-5.
2. **Activare:** AlexNet a folosit **ReLU** (Rectified Linear Unit) în loc de funcția **Sigmoid**. (Gradientul ReLU este întotdeauna 1 pe intervalul pozitiv, prevenind dispariția gradientului).
3. **Dimensiuni Kernel:** Primul strat folosește o fereastră convoluțională mare de 11×11 (pentru a capta obiecte pe imaginile ImageNet, care sunt de peste zece ori mai mari decât imaginile MNIST ale LeNet). Straturile următoare folosesc 5×5, urmate de 3×3.
4. **Pooling:** AlexNet folosește **Maximum Pooling** (3×3 kernel cu *stride* 2) după primul, al doilea și al cincilea strat convoluțional.
5. **Canale:** AlexNet folosește **de zece ori mai multe canale** de convoluție decât LeNet.
6. **Regularizare:** AlexNet controlează complexitatea stratului FC folosind **Dropout**, pe când LeNet folosea doar *weight decay*.
7. **Augmentare:** A utilizat **augmentarea masivă a imaginii** (flipping, clipping, color changes) pentru a face modelul mai robust și pentru a reduce *overfitting*-ul.
8. **Straturi FC:** Cele două straturi complet conectate au **4096 de ieșiri** fiecare și produc aproape **1 GB de parametri**.

Caracteristici Ierarhice Învățate

- Modelele profunde învață **reprezentări ierarhice**.
- Straturile inferioare învață **filtre de extragere a caracteristicilor** care seamănă cu filtrele tradiționale (margini, culori).
- Straturile superioare construiesc structuri mai mari (ochi, nas, lame de iarbă) și, în cele din urmă, obiecte întregi (oameni, câini, avioane).

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: ReLU vs. Sigmoid

- **Explicație Simplificată (Gradientul Vanishing):** Funcția Sigmoid aplatizează ieșirile către 0 sau 1 pentru valori mari. Când o ieșire este foarte aproape de 0 sau 1, panta (gradientul) este aproape zero. Dacă gradientul este zero, *backpropagation*-ul nu poate actualiza parametrii modelului, iar învățarea se oprește (fenomenul de *vanishing gradient*).
- **Avantajul ReLU:** ReLU este $f(x) = \max(0, x)$. Pentru orice valoare pozitivă, gradientul este **întotdeauna 1**. Acest lucru asigură că, atâta timp cât ieșirea este pozitivă, gradientul curge eficient înapoi prin rețea, făcând antrenarea modelelor adânci mult mai ușoară.

Concept Dificil: 11×11 Kernel în Primul Strat

- **Context:** De ce AlexNet folosește un kernel atât de mare (spre deosebire de kernelurile 3×3 preferate astăzi)?
- *Asta se întâmplă deoarece:* Imaginile ImageNet (224×224 pixeli) sunt mult mai mari decât imaginile MNIST (28×28). Un kernel mare era necesar în primul strat pentru a avea un **câmp receptiv suficient de mare** pentru a capta obiecte pe o zonă extinsă, înainte ca *pooling*-ul să reducă drastic dimensiunea.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Hardware și Optimizare):**
  - **CAUZĂ:** Convoluțiile și înmulțirile matriciale sunt operații care pot fi **paralelizate** pe hardware.
  - **EFECT:** Dezvoltarea GPU-urilor și realizarea că acestea erau perfecte pentru calculul convoluțional (fiind optimizate pentru produse matriciale 4×4) au făcut antrenarea rețelelor adânci posibilă din punct de vedere computațional.
- **Implicația Paradigmei:** Triumful AlexNet a marcat sfârșitul erei *feature engineering*-ului manual și trecerea la sisteme **end-to-end** bazate pe învățarea ierarhică a reprezentărilor vizuale.

---

5. EXERCITII (Calcul pe Arhitectura AlexNet)

Vom genera exerciții de calcul concentrate pe dimensiunile masive și parametrii specifici ai AlexNet.

Exercițiul A.1: Analiza Fluxului Dimensional (Stratul C1)

Analizați primul strat convoluțional al AlexNet (versiune simplificată):

- Intrare: Imagine 224×224, 3 canale ($C_{in} = 3$).
- Strat C1: 11×11 Conv, 96 ieșiri ($C_{out} = 96$).
- Stride: 4. Padding: 0.

Calculați dimensiunea spațială a ieșirii (Înălțime $\times$ Lățime).

Formula dimensională:

$$n'_h = \lfloor (n_h - k_h + p_h)/s_h \rfloor + 1$$

Rezolvare A.1:

1. **Calculul Înălțimii ieșirii (n'_h):**

$$n'_h = \lfloor (224 - 11 + 0)/4 \rfloor + 1$$
$$n'_h = \lfloor 213/4 \rfloor + 1 = 53 + 1 = 54$$

2. **Calculul Lățimii ieșirii (n'_w):**

$$n'_w = 54$$

3. **Dimensiunea ieșirii:** $96@\mathbf{54} \times \mathbf{54}$.

Exercițiul A.2: Calculul Numărului de Parametri (Stratul C1)

Calculați numărul total de parametri reali (Greutăți V și Bias-uri u) pentru Stratul C1 al AlexNet.

- $C_{in} = 3, C_{out} = 96, k = 11 \times 11$.

Rezolvare A.2:

1. **Greutăți (V):** $C_{out} \times C_{in} \times k_h \times k_w$

$$V = 96 \times 3 \times 11 \times 11 = 288 \times 121 = \mathbf{34,848}$$

2. **Bias-uri (u):** $C_{out} = \mathbf{96}$
3. **Total Parametri C1:** $34,848 + 96 = \mathbf{34,944}$ parametri.

Exercițiul A.3: Calculul Parametrilor (Stratul Fully Connected - FC)

AlexNet conține straturi FC masive. Considerați unul dintre straturile FC ascunse:

- Intrare: 4096 unități.
- Ieșire: 4096 unități.

Calculați numărul total de parametri pentru acest strat FC.

Rezolvare A.3:

1. **Ponderi (W):** $N_{\text{in}} \times N_{\text{out}}$

$$W = 4096 \times 4096 = 16,777,216$$

2. **Bias-uri (b):** $N_{\text{out}} = 4,096$
3. **Total Parametri FC:** $16,777,216 + 4,096 = \mathbf{16,781,312}$ parametri.

(Acest număr mare ilustrează de ce AlexNet avea aproape 1 GB de parametri și necesita optimizări hardware, conform sursei.)

3.8

Absolut! Vă prezint Ghidul de Studiu Complet pentru lecția **3.8 Rețele bazate pe blocuri (VGG)**, o etapă crucială care simplifică și standardizează designul arhitecturilor neuronale profunde.

Ghid de Studiu Complet: 3.8 Rețele bazate pe blocuri (VGG)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.8 explorează evoluția designului rețelelor neuronale de la structuri ad-hoc la **arhitecturi modulare**, introducând conceptul de **blocuri (blocks)** de rețea, un concept popularizat de rețeaua **VGG** (Visual Geometry Group). Deși **AlexNet** a oferit dovezi empirice că CNN-urile profunde pot obține rezultate bune, nu a furnizat un șablon general (*general template*) pentru proiectarea rețelelor ulterioare. Designul rețelelor neuronale a evoluat, reflectând progresul din ingineria electronică (de la tranzistori la elemente logice și apoi la **blocuri logice**). Această trecere de la gândirea în termeni de neuroni individuali la straturi întregi, și acum la **blocuri care repetă tipare de straturi**, a făcut proiectarea rețelelor profunde mult mai abstractă și mai ușor de implementat prin utilizarea buclelor și subrutinelor în limbajele de programare moderne.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Abstracție:** Designul arhitecturii a evoluat de la straturi individuale la **blocuri, repetând tipare de straturi**.
2. **Originea VGG:** Ideea utilizării blocurilor a apărut din grupul **Visual Geometry Group (VGG)** de la Universitatea Oxford, care a dat numele rețelei lor VGG.

3. **Structura Blocului VGG Clasică:** Blocul VGG constă dintr-o secvență de straturi convoluționale urmată de un **strat de *maximum pooling* pentru *downsampling* spațial**.
4. **Componentele de Bază:** Un bloc de bază al CNN-urilor clasice conține: un strat convoluțional cu *padding* pentru menținerea rezoluției, o neliniaritate (precum **ReLU**), și un strat de *pooling* (precum *maximum pooling*).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.8 Rețele bazate pe blocuri (VGG)

- **Necesitatea Șablonului:** AlexNet a dovedit puterea CNN-urilor profunde, dar nu a oferit un șablon general (*general template*) pentru a ghida cercetătorii în proiectarea rețelelor noi.
- **Evoluția Designului:** Progresul în designul rețelelor neuronale oglindește progresul în designul cipurilor, unde inginerii au trecut de la tranzistori la blocuri logice. Similar, designul rețelelor a devenit mai abstract, trecând la **blocuri și tipare repetitive de straturi**.
- **Implementare:** Aceste structuri repetitive sunt ușor de implementat în cod, în cadrul oricărui cadru modern de *deep learning*, folosind **bucle și subrutine**.

Structura Blocului VGG

- **Blocul de Bază (Classic CNN):** Blocul fundamental de construire al CNN-urilor clasice este o secvență care include:
  - Un **strat convoluțional** cu *padding* pentru menținerea rezoluției.
  - O **neliniaritate** (ex: ReLU).
  - Un **strat de *pooling*** (ex: *maximum pooling*).
- **Blocul VGG Specific:** Un bloc VGG este compus dintr-o secvență de straturi convoluționale, urmate de un **strat de *maximum pooling*** pentru *downsampling* spațial.

Detalii Arhitecturale VGG Originale

- În lucrarea VGG originală, autorii au folosit:
  - **Convoluții cu kernel de** 3×3 și **padding de 1** (pentru a menține înălțimea și lățimea constante).
  - **Maximum pooling de** 2×2 cu **stride de 2** (pentru a înjumătăți rezoluția după fiecare bloc).

Structura Generală VGG (Similaritate cu LeNet și AlexNet)

- Rețeaua VGG poate fi împărțită în două părți, la fel ca LeNet și AlexNet:
  1. Prima parte constă în principal din **straturi convoluționale și de *pooling***.

2. A doua parte constă din **straturi complet conectate (Fully-Connected)**.
- Partea complet conectată a rețelei VGG este **identică** cu cea acoperită în AlexNet (deși imaginea 13 arată o versiune cu 4096, 4096 și 1000 de ieșiri).

VGG-11 (Exemplu Specific)

- Rețeaua VGG originală avea **cinci blocuri convoluționale**.
- Primele două blocuri aveau **un singur strat convoluțional** fiecare.
- Ultimele trei blocuri conțineau **două straturi convoluționale** fiecare.
- **Canalele de leșire:** Primul bloc are **64 de canale de ieșire**, iar fiecare bloc ulterior **dublează numărul de canale de ieșire**, până când se ajunge la **512**.
- Deoarece această rețea folosește **8 straturi convoluționale** și **3 straturi complet conectate**, este denumită adesea **VGG-11**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Bloc (VGG Block)

- **Analogie Simplificată (LEGO):** Blocurile VGG sunt ca piesele LEGO. În loc să proiectezi fiecare piesă (strat) individual, folosești un model standardizat (Conv-Conv-Pool sau Conv-Pool) pe care îl poți repeta de mai multe ori. Acest lucru accelerează proiectarea și face arhitectura mai ușor de scalat (poți face rețeaua mai adâncă adăugând mai multe blocuri, fără a reinventa structura fiecărui strat).

Concept Dificil: 3×3 Convoluții cu Padding 1

- **Context:** Sursa menționează că VGG folosește 3×3 kerneluri cu *padding* 1 pentru a menține rezoluția.
- *Asta se întâmplă deoarece:* Formula pentru ieșirea spațială (fără stride) este $(n - k + 2p + 1)$. Dacă $k = 3$ și $p = 1$: $(n - 3 + 2 \cdot 1 + 1) = n$. Prin urmare, 3×3 cu *padding* 1 este cea mai mică și mai eficientă operație care **menține rezoluția spațială neschimbată** între straturile convoluționale, permițând acumularea de adâncime a canalului înainte de *downsampling*-ul prin *pooling*.

Downsampling Spațial prin Pooling

- *Asta se întâmplă deoarece:* Deși straturile convoluționale din interiorul blocului VGG mențin rezoluția, *maximum pooling*-ul de 2×2 cu *stride* 2, aplicat la finalul blocului, este cel care **înjumătățește** dimensiunea spațială. Acest lucru este esențial pentru ca rețeaua să poată detecta caracteristici la rezoluții din ce în ce mai mari (un câmp receptiv mai mare).

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu AlexNet (Lecția 3.7):** VGG a preluat succesul lui AlexNet, dar a îmbunătățit designul, făcându-l mai **sistematic** și mai **ușor de scalat**. În plus, VGG a păstrat structura *fully-connected* a AlexNet (FC 4096, FC 4096, FC 1000).
- **Relația cu Convoluția 1×1 (Lecția 3.4):** Deși nu este direct menționat că VGG folosește 1×1 (preferând 3×3), conceptul de a controla fluxul de canale (dublarea C_{out} la fiecare bloc) este fundamental, similar cu funcția de transformare a canalelor realizată de 1×1 convoluții.

---

5. EXERCITII (Calcul pe Baza Arhitecturii VGG)

Voi genera exerciții de calcul bazate pe structura blocurilor VGG și pe calculul de parametri/dimensiuni, folosind modelul detaliat de rezolvare din surse.

Exercițiul V.1: Analiza Fluxului Dimensional (Bloc VGG Standard)

Considerați un bloc VGG standard format dintr-un strat convoluțional, urmat de ReLU și un strat de *maximum pooling*.

- Intrare: $n_h = 32$, $n_w = 32$. Canale $C_{in} = 64$.
- Strat Conv: 3×3 kernel, 128 canale de ieșire ($C_{out} = 128$), Padding 1, Stride 1.
- Strat MaxPool: 2×2 kernel, Stride 2.

1. Calculați dimensiunea tensorului de ieșire (Canale @ Înălțime $\times$ Lățime) după stratul Conv.
2. Calculați dimensiunea finală a tensorului de ieșire după stratul MaxPool.

Rezolvare V.1:

1. **Ieșirea Stratului Conv (C1):**
   - **Canale:** $C_{out} = 128$.
   - **Înălțimea/Lățimea:** Formula cu Padding 1 și 3×3 (Stride 1):

$$n'_h = (n_h - k_h + 2p + 1) = (32 - 3 + 2 \cdot 1 + 1) = 32$$

   - Dimensiunea C1: $\mathbf{128@32 \times 32}$.
2. **Ieșirea Stratului MaxPool (S2):**
   - **Canale:** *Pooling*-ul nu schimbă numărul de canale, deci $C_{out} = 128$.
   - **Înălțimea/Lățimea:** Formula pentru 2×2 kernel, Stride 2: $32/2 = 16$.
   - Dimensiunea S2: $\mathbf{128@16 \times 16}$.

Exercițiul V.2: Calculul Numărului de Parametri (Strat Convoluțional VGG)

Calculați numărul total de parametri reali (Greutăți V și Bias-uri u) pentru stratul convoluțional din Exercițiul V.1.

- $C_{in} = 64, C_{out} = 128, k = 3 \times 3.$

Rezolvare V.2:

1. **Greutăți (V):** $C_{out} \times C_{in} \times k_h \times k_w$

$$V = 128 \times 64 \times 3 \times 3$$
$$V = 128 \times 64 \times 9 = 8192 \times 9 = \mathbf{73,728}$$

2. **Bias-uri (u):** $C_{out} = \mathbf{128}$
3. **Total Parametri:** $73,728 + 128 = \mathbf{73,856}$ parametri.

Exercițiul V.3: Calculul Ieșirii (Multi-Channel Cross-Correlation)

Acest exercițiu simulează o singură operație de cross-correlation, urmând structura detaliată de calcul din sursa, dar cu numere mai simple.

Considerați un strat convoluțional cu $C_{in} = 2$, $C_{out} = 1$ și $k_h \times k_w = 2 \times 2$. Presupunem că nu există padding și stride 1. Calculați elementul de ieșire $H_{0,0,1}$.

- Bias-ul canalului de ieșire 1 este $u_1 = 5$.

$$\text{Intrări (patc } 2 \times 2\text{): } X_{c=1} = [1 \quad 1\ 1 \quad 1], \quad X_{c=2} = [2 \quad 2\ 2 \quad 2]$$
$$\text{ernel (pentru analul de Ieșire 1): } V_{,,c=1,d=1} = [1 \quad 0\ 0 \quad 1], \quad V_{,,c=2,d=1} = [1 \quad 1\ 1 \quad 1]$$

Rezolvare V.3:

1. **Contribuția din Canalul de Intrare 1 ($c = 1$):**

$$\text{Suma}_1 = (1 \cdot 1) + (0 \cdot 1) + (0 \cdot 1) + (1 \cdot 1) = 1 + 0 + 0 + 1 = 2$$

2. **Contribuția din Canalul de Intrare 2 ($c = 2$):**

$$\text{Suma}_2 = (1 \cdot 2) + (1 \cdot 2) + (1 \cdot 2) + (1 \cdot 2) = 2 + 2 + 2 + 2 = 8$$

3. **Adăugarea Bias-ului și Sumarea Contribuțiilor:**

$$[\mathbf{H}]_{0,0,1} = u_1 + \text{Suma}_1 + \text{Suma}_2$$
$$[\mathbf{H}]_{0,0,1} = 5 + 2 + 8 = \mathbf{15}$$

3.9

Acest ghid de studiu se concentrează pe **3.9 Normalizarea pe Mini-Batch (Batch Normalization - BN)**, o tehnică fundamentală care a permis construirea și antrenarea eficientă a rețelelor neuronale profunde.

Ghid de Studiu Complet: 3.9 Normalizarea pe Mini-Batch (Batch Normalization)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Antrenarea rețelelor neuronale profunde este notorie ca fiind dificilă, iar obținerea convergenței într-un timp rezonabil poate fi complicată. Lecția 3.9 introduce **Normalizarea pe Mini-Batch (BN)**, o tehnică populară și eficientă care **accelerează consistent convergența** rețelelor profunde. Problema majoră pe care BN o abordează este *drift*-ul distribuției variabilelor (*widely varying magnitudes*) în straturile intermediare pe măsură ce parametrii modelului sunt actualizați. BN rezolvă această problemă prin **standardizarea** activărilor la fiecare strat, bazată pe statisticile mini-batch-ului curent (media și deviația standard), menținând astfel mărimile variabilelor centrate și rescalate. BN, alături de blocurile reziduale (discutate în Secțiunea 3.10), a făcut posibilă antrenarea de rutină a rețelelor cu **peste 100 de straturi**.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Scopul Principal:** BN accelerează antrenarea rețelelor profunde, prevenind în special **divergența mărimilor variabilelor** în straturile intermediare.
2. **Mecanismul de Bază:** BN normalizează intrările fiecărui strat prin **scăderea mediei** și **împărțirea la deviația standard**, ambele calculate din **mini-batch-ul curent**.
3. **Parametrii Învățați (Gamma și Beta):** După standardizare, se aplică parametrii învățați γ (coeficient de scară) și β (deplasare/offset de scară) pentru a permite rețelei să ajusteze distribuția.
4. **Moduri de Operare:** BN operează diferit în **modul de antrenare** (folosind statistici zgomotoase ale mini-batch-ului) și în **modul de predicție** (folosind statistici stabile calculate pe întregul set de date).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.9 Batch Normalization (Normalizarea pe Mini-Batch)

Probleme Practice și Motivația BN

- **Preprocesarea Datelor (Analogia):** Prima etapă în lucrul cu datele este **standardizarea** caracteristicilor de intrare pentru a avea o **medie de zero** și o **varianță**

de unu. Intuitiv, această standardizare ajută optimizatorii, deoarece plasează parametrii *a priori* la o scară similară.

- **Problema Mărimilor Variabile:** Pentru un MLP sau CNN, ieșirile transformărilor afine din straturile intermediare pot lua valori cu **mărimi larg variabile**. Această variație apare de-a lungul straturilor, între unitățile din același strat și de-a lungul timpului de antrenare, din cauza actualizărilor parametrilor.
- **Divergența Rețelei:** Inventatorii BN au postulat informal că această "drift" în distribuția variabilelor ar putea **împiedica convergența rețelei**. Un strat cu valori de 100 de ori mai mari decât altul ar putea necesita ajustări compensatorii în ratele de învățare.
- **Regularizare:** Rețelele mai profunde sunt mai complexe și mai susceptibile la *overfitting*, făcând ca **regularizarea să fie critică**.

Mecanismul BN (Formal)

- BN se aplică straturilor individuale.
- În fiecare iterație de antrenare, se normalizează intrările (ale BN) prin:
  1. **Scăderea mediei**.
  2. **Împărțirea la deviația standard**. Ambele sunt estimate pe baza statisticilor **mini-batch-ului curent** ($\mathcal{B}$).
- Această normalizare bazată pe statistici de mini-batch dă numele metodei.
- După standardizare, mini-batch-ul rezultat are **media zero** și **varianța unitară**.
- Se aplică apoi un **coeficient de scară** γ și o **deplasare de scară** β.
- γ și β sunt element-wise, au aceeași formă ca intrarea $\mathbf{x}$ și sunt **parametri învățați** împreună cu ceilalți parametri ai modelului.

Formula Formală (Ecuația 4)

$$\mathrm{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta \quad (4)$$

- Media eșantionului: $\hat{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x}$.
- Varianța eșantionului (plus epsilon): $\hat{\sigma}^2_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2 + \epsilon$.
- Se adaugă o constantă mică $\epsilon > 0$ la estimarea varianței pentru a preveni **împărțirea la zero**.

Beneficii și Implicații

- BN centrează și rescallează activările înapoi la o medie și mărime date, prevenind astfel **divergența mărimilor variabile** în straturile intermediare.
- BN permite rate de învățare mai agresive.
- Estimările zgomotoase ale mediei și varianței bazate pe mini-batch-uri sunt de fapt benefice.

Cerința Mini-Batch-ului

- Dacă BN ar fi aplicat cu mini-batch-uri de mărime 1, nu s-ar putea învăța nimic.
- Prin scăderea mediei, fiecare unitate ascunsă ar lua valoarea 0.
- BN necesită mini-batch-uri suficient de mari pentru a fi eficientă și stabilă.

Aplicarea BN în Arhitecturi

BN operează pe un mini-batch complet la un moment dat și nu poate ignora dimensiunea batch-ului.

- **Straturi Complet Conectate (FC):** BN este inserat **după transformarea afină ($W\mathbf{x} + \mathbf{b}$) și înainte de funcția de activare neliniară (ϕ)**.

$$\mathbf{h} = \phi(\mathrm{BN}(W\mathbf{x} + \mathbf{b}))$$

- **Straturi Convoluționale:** BN este aplicat similar: **după convoluție și înainte de funcția de activare neliniară**.
  - Când convoluția are **canale de ieșire multiple**, BN trebuie efectuat pentru **fiecare canal**.
  - Fiecare canal are propriul coeficient de scară (γ) și deplasare (β).
  - Normalizarea se efectuează peste $m \cdot p \cdot q$ elemente per canal de ieșire (unde $m =$ mărimea batch-ului, $p, q =$ înălțimea/lățimea). Același γ și β sunt folosite pentru toate locațiile spațiale dintr-un canal dat.

Modul de Antrenare vs. Modul de Predicție

- BN se comportă diferit în modul de antrenare și cel de predicție.
- **Modul de Antrenare:** Folosește statistici bazate pe mini-batch-uri (care sunt zgomotoase, dar utile).
- **Modul de Predicție/Testare:** Zgomotul din statisticile eșantionului nu mai este dorit. După antrenare, se folosește întregul set de date pentru a calcula **estimări stabile** ale mediei și varianței, care sunt **fixate** și folosite în timpul predicției.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Element-Wise Scale Parameter (γ) și Shift Parameter (β)

- **Explicație Simplificată (Ajustarea Tonului):** BN forțează activările să aibă media zero și varianța unu. Totuși, rețeaua ar putea funcționa optim dacă, de exemplu, activările ar avea media 0.5 și varianța 2.
- *Asta se întâmplă deoarece:* Parametrii γ și β permit rețelei să "împingă" distribuția înapoi la o formă ne-standardizată, dacă acest lucru ajută la reducerea erorii. γ (scară) controlează *cât de mult* se întinde distribuția, iar β (deplasare) controlează *unde* este

centrată. Acestea sunt învățate pentru a optimiza performanța, în timp ce BN menține distribuția stabilă în timpul antrenării.

Concept Dificil: De ce Mini-Batch-urile de Mărime 1 Eșuează

- **Analogie:** Imaginați-vă că aveți un singur număr (batch size = 1), să zicem 5. Media acestui batch este 5. Dacă scădeți media (5 - 5), rezultatul este 0. Varianța este de asemenea 0. Indiferent de valoarea de intrare, dacă batch-ul este 1, ieșirea normalizată va fi întotdeauna zero, iar antrenarea nu poate avea loc.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Inițializarea Parametrilor (Lecția 2.7):** La fel cum inițializarea bună este crucială pentru a începe antrenarea, BN asigură că distribuția rămâne stabilă *pe parcursul* antrenării, eliminând necesitatea unor ajustări manuale constante ale ratelor de învățare.
- **Implicația asupra Adâncimii:** Capacitatea BN de a menține stabile mărimile variabilelor este motivul principal pentru care, împreună cu blocurile reziduale, a făcut posibilă antrenarea de rutină a rețelelor cu **peste 100 de straturi**.
- **Conexiunea cu Regularizarea:** Deoarece BN introduce un zgomot (prin utilizarea statisticilor bazate pe mini-batch), acesta acționează ca o formă subtilă de **regularizare**, similară cu *dropout* (care, de asemenea, se comportă diferit în modul de antrenare și predicție).

---

5. EXERCITII (Calcul pe Normalizarea pe Mini-Batch)

Vom genera exerciții de calcul concentrate pe aspectele matematice ale BN, folosind formulele furnizate în surse.

Exercițiul B.4: Calculul Standardizării (Mini-Batch 1D)

Un strat FC primește un mini-batch $\mathcal{B}$ de 3 exemple, fiecare având o singură unitate ascunsă, $\mathbf{x} \in \mathbb{R}^1$:

$$\mathcal{B} = x_1 = 10, x_2 = 15, x_3 = 20$$

1. Calculați media eșantionului $\hat{\mu}_\mathcal{B}$.
2. Calculați varianța eșantionului $\hat{\sigma}^2_\mathcal{B}$ (Presupuneți $\epsilon = 0$).
3. Calculați valoarea standardizată (ieșirea $\frac{\mathbf{x}-\hat{\mu}_\mathcal{B}}{\hat{\sigma}_\mathcal{B}}$) pentru elementul $x_1 = 10$.

Rezolvare B.4:

1. **Calculul Mediei ($\hat{\mu}_{\mathcal{B}}$):**

$$\hat{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x} = \frac{10 + 15 + 20}{3} = \frac{45}{3} = \mathbf{15}$$

2. **Calculul Varianței ($\hat{\sigma}^2_{\mathcal{B}}$):**

$$\hat{\sigma}^2_{\mathcal{B}} = \frac{1}{3}\left[(10-15)^2 + (15-15)^2 + (20-15)^2\right]$$

$$\hat{\sigma}^2_{\mathcal{B}} = \frac{1}{3}\left[(-5)^2 + 0^2 + 5^2\right] = \frac{25 + 0 + 25}{3} = \frac{50}{3} \approx \mathbf{16.67}$$

   Deviația standard $\hat{\sigma}_{\mathcal{B}} = \sqrt{50/3} \approx 4.0825$

3. **Calculul Ieșirii Standardizate pentru $x_1 = 10$:**

$$x'_1 = \frac{x_1 - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} = \frac{10 - 15}{4.0825} = \frac{-5}{4.0825} \approx \mathbf{-1.2247}$$

Exercițiul B.5: Aplicarea Parametrilor Învățați (γ și β)

Continuând de la Exercițiul B.4, aplicați BN completă pentru elementul $x_1 = 10$, folosind următoarele valori învățate:

- Coeficient de Scară $\gamma = 4$.
- Deplasare de Scară $\beta = 1$.

Formula BN: $\mathrm{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta$

Rezolvare B.5:

$$\mathrm{BN}(x_1) = \gamma \cdot x'_1 + \beta$$

$$\mathrm{BN}(x_1) = 4 \cdot (-1.2247) + 1$$

$$\mathrm{BN}(x_1) = -4.8988 + 1 = \mathbf{-3.8988}$$

Exercițiul B.6: Calculul Numărului de Parametri BN (Strat Convoluțional)

Considerați un strat convoluțional cu 128 de canale de ieșire ($C_{out} = 128$).

1. Câți parametri totali (învățați) de Normalizare pe Mini-Batch (BN) sunt adăugați de acest strat (ignorând greutățile și bias-urile convoluționale V și u)?
2. Explicați de ce acești parametri sunt **scalați per canal** și nu per pixel.

Rezolvare B.6:

1. **Numărul Total de Parametri BN:** Stratul BN introduce un coeficient de scară γ și o deplasare de scară β **pentru fiecare canal de ieșire**.

$$\text{Parametri} = C_{out}(\text{pentru } \gamma) + C_{out}(\text{pentru } \beta)$$

$$\text{Parametri} = 128 + 128 = \mathbf{256}$$

2. **Scalarea per Canal:** Pentru straturile convoluționale, BN normalizează valoarea la fiecare locație spațială (p, q) folosind **aceeași** medie și varianță (estimată pe toate $m \cdot p \cdot q$ elemente) pentru un canal dat. Astfel, γ și β sunt elemente scalare aplicate întregului *feature map* al acelui canal. Aceasta menține principiul **invarianței la translație** specifică CNN-urilor, deoarece transformarea aplicată (normalizarea) nu depinde de locația spațială.

3.10

Absolut! Ghidul de studiu complet pentru lecția **3.10 Rețele Reziduale (ResNet)** este gata. Această lecție prezintă o inovație arhitecturală fundamentală care a permis construirea rețelelor neuronale extrem de adânci.

Ghid de Studiu Complet: 3.10 Rețele Reziduale (ResNet)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 3.10 se concentrează pe **Rețeaua Reziduală (ResNet)**, o arhitectură inovatoare propusă de He et al., care a avut o influență profundă asupra modului în care sunt construite rețelele neuronale profunde. Pe măsură ce rețelele devin din ce în ce mai adânci, apare o provocare critică: cum ne asigurăm că adăugarea de straturi suplimentare nu doar mărește complexitatea, ci face rețeaua **strict mai expresivă**. Soluția propusă de ResNet este surprinzător de simplă: **blocul rezidual (*residual block*)**. Ideea de bază este că fiecare strat adițional ar trebui să poată conține mai ușor **funcția de identitate** (*identity function*) ca unul dintre elementele sale. Această capacitate este facilitată de **conexiunile reziduale** (*residual connections*) sau **conexiunile shortcut** (*shortcut connection*), care permit informației să se propage mai rapid înainte, prin salturi peste straturi. ResNet a câștigat ImageNet Large Scale Visual Recognition Challenge în **2015**.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Obiectivul Adâncimii:** Să ne asigurăm că adăugarea de straturi face rețeaua **strict mai expresivă**.
2. **Inima ResNet (Funcția Identitate):** Principiul central este că un bloc nou trebuie să poată învăța cu ușurință **funcția de identitate** ($f(\mathbf{x}) = \mathbf{x}$).
3. **Blocul Rezidual:** Blocurile standard învață funcția de mapare $f(\mathbf{x})$. Blocul rezidual învață maparea reziduală $f(\mathbf{x}) - \mathbf{x}$. Dacă maparea optimă este identitatea, este mai ușor să "împingi" ponderile și bias-urile stratului superior la zero pentru a învăța zero (reziduul) decât să forțezi maparea directă la identitate.

4. **Conexiunea Shortcut:** Conexiunile reziduale transportă intrarea $\mathbf{x}$ direct la operatorul de adunare, permițând propagarea rapidă a informației prin straturi.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

3.10 Rețele Reziduale (ResNet)

Diferența dintre Blocurile Standard și Blocurile Reziduale

- Focalizarea se face pe o parte locală a rețelei neuronale, cu intrarea notată $\mathbf{x}$.
- **Blocul standard (Normal Block):** Porțiunea din cutia cu linii punctate învață direct maparea dorită $f(\mathbf{x})$, care este folosită ca intrare în funcția de activare finală.
- **Blocul Rezidual (Residual Block):** Porțiunea din cutia cu linii punctate învață **maparea reziduală** $f(\mathbf{x}) - \mathbf{x}$, de unde își derivă numele.
- **Conexiunea Reziduală (*Residual connection* / *shortcut connection*):** O linie continuă care transportă intrarea stratului $\mathbf{x}$ direct la operatorul de adunare.
- **Învățarea Funcției Identitate:** Dacă maparea dorită $f(\mathbf{x})$ este funcția identitate ($f(\mathbf{x}) = \mathbf{x}$), maparea reziduală $f(\mathbf{x}) - \mathbf{x}$ devine **mai ușor de învățat**. Este necesară doar forțarea ponderilor și bias-urilor stratului superior (ex: stratul complet conectat sau convoluțional) din blocul cu linii punctate să fie zero.

Arhitectura Blocului ResNet

- ResNet urmează designul VGG de a utiliza în principal convoluții 3×3.
- Blocul rezidual tipic conține **două straturi convoluționale** 3×3 cu același număr de canale de ieșire.
- Fiecare strat convoluțional este urmat de un strat de **Normalizare pe Mini-Batch (BN)** și o funcție de activare **ReLU**.
- Intrarea $\mathbf{x}$ ocolește (*skips*) cele două operații de convoluție și este adunată direct înainte de funcția de activare ReLU finală.
- Designul necesită ca ieșirea celor două straturi convoluționale să aibă **aceeași formă** ca intrarea $\mathbf{x}$, pentru a putea fi adunate (operație element-wise).

Ajustarea Canalele și Rezoluției

- Dacă se dorește modificarea numărului de canale sau a rezoluției (înălțime/lățime), este necesară introducerea unui strat convoluțional adițional de 1×1.
- Există, așadar, **două tipuri de blocuri ResNet**:
  1. **Blocul Standard (Fără 1×1 Conv):** Adaugă direct intrarea la ieșire (când canalele și rezoluția sunt aceleași).
  2. **Blocul de Proiecție (Cu 1×1 Conv):** Ajustează canalele și/sau rezoluția folosind un strat 1×1 convoluțional pe *shortcut connection* înainte de adunare.

Structura Globală a Arhitecturii ResNet-18

- ResNet folosește **patru module** compuse din blocuri reziduale.
- Rețeaua începe cu:
  - Un strat convoluțional 7×7 cu 64 de canale de ieșire și *stride* 2.
  - Urmat de un strat de *maximum pooling* 3×3 cu *stride* 2.
  - Un strat BN este adăugat după fiecare strat convoluțional.
- Numărul de canale în primul modul este același cu cel al intrării.
- În primul bloc rezidual al fiecărui modul subsecvent, **numărul de canale este dublat**, iar **înălțimea și lățimea sunt înjumătățite**.
- Se adaugă un strat final de **Global Average Pooling**, urmat de stratul de ieșire complet conectat (FC).
- **ResNet-18:** Modelul standard include 4 straturi convoluționale în fiecare modul (excluzând convoluția 1×1). Împreună cu primul strat 7×7 și stratul FC final, modelul are **18 straturi în total**.
- Arhitectura simplă și ușor de modificat a ResNet a dus la utilizarea sa rapidă și larg răspândită, permițând crearea de modele mai adânci, cum ar fi **ResNet-152**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Maparea Reziduală ($f(\mathbf{x}) - \mathbf{x}$)

- **Explicație Simplificată (Ajustarea Preciziei):** Imaginați-vă că un strat a învățat deja o funcție excelentă $f(\mathbf{x})$ care este aproape perfectă (de exemplu, $f(\mathbf{x}) = \mathbf{x} + 0.001$). Un strat normal ar trebui să învețe $f(\mathbf{x})$ de la zero. Un bloc rezidual trebuie să învețe doar eroarea rămasă, **reziduul** (adică 0.001).
- *Asta se întâmplă deoarece:* Învățarea unei mici ajustări (zero, dacă nu este necesară nicio îmbunătățire) este mult mai ușoară pentru optimizator decât încercarea de a învăța funcția identitate (sau o funcție similară) prin ponderi mari și greoaie.

Concept Dificil: Rolul Convoluției 1×1 în Blocul de Proiecție

- **Context:** Convoluția 1×1 este plasată pe conexiunea *shortcut* atunci când dimensiunea intrării $\mathbf{x}$ nu se potrivește cu ieșirea celorlalte două straturi (de exemplu, dacă numărul de canale s-a dublat).
- *Asta se întâmplă deoarece:* Operația de adunare (plus) dintre intrarea $\mathbf{x}$ și ieșirea $f(\mathbf{x}) - \mathbf{x}$ este o operație element-wise. Aceasta necesită ca cele două tensoare să aibă exact **aceeași formă spațială și același număr de canale**. Convoluția 1×1 este instrumentul ideal, deoarece poate schimba numărul de canale și, dacă i se dă un *stride* > 1, poate reduce rezoluția spațială, asigurând alinierea formei.

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația cu Normalizarea pe Mini-Batch (Lecția 3.9):** BN este o componentă esențială în blocurile ResNet. Fără BN pentru a stabiliza activările, antrenarea rețelelor cu 18, 50 sau 152 de straturi (cum ar fi ResNet-152) ar fi extrem de dificilă sau chiar imposibilă.
- **Relația cu VGG (Lecția 3.8):** ResNet adoptă filosofia VGG de a folosi kerneluri mici, repetitive 3×3. Simplificarea și modularitatea ResNet continuă tendința stabilită de VGG.
- **Implicația asupra Propagării Înapoi (*Backpropagation*):** Conexiunile *shortcut* nu numai că ajută la propagarea rapidă a informației înainte, dar oferă și o cale directă pentru propagarea gradientului înapoi, prevenind fenomenul de *vanishing gradient* în rețelele foarte adânci.

---

5. EXERCITII (Calcul pe Arhitectura ResNet și Convoluții)

Voi genera exerciții de calcul concentrate pe aspectele ResNet și pe formulele generale de convoluție utilizate de-a lungul capitolului.

Exercițiul R.1: Calculul Fluxului Dimensional (Stratul Inițial ResNet)

Analizați primele două straturi ale arhitecturii ResNet-18:

- Intrare: Imagine 224×224, 3 canale ($C_{in} = 3$).
- Strat C1: 7×7 Conv, 64 ieșiri ($C_{out} = 64$), Stride 2. (Presupunem Padding 3 pentru a păstra dimensiunea spatială după *pooling*).
- Strat MaxPool: 3×3 MaxPool, Stride 2.

Calculați dimensiunea spațială a ieșirii (Înălțime $\times$ Lățime) după stratul C1 și după stratul MaxPool.

Formula dimensională (simplificată pentru padding p): $n' = \lfloor (n - k + 2p)/s \rfloor + 1$

Rezolvare R.1:

1. **Ieșirea Stratului C1 (7×7 Conv, Stride 2, Padding 3):**

   $$n'_h = \lfloor (224 - 7 + 2 \cdot 3)/2 \rfloor + 1 = \lfloor (223)/2 \rfloor + 1 = 111 + 1 = 112$$

   Dimensiunea C1: $64@112 \times 112$.
2. **Ieșirea Stratului MaxPool (3×3 Pool, Stride 2, Padding 0):** Folosind intrarea de 112×112:

$$n'_h = \lfloor (112 - 3 + 0)/2 \rfloor + 1 = \lfloor 109/2 \rfloor + 1 = 54 + 1 = 55$$

Dimensiunea Finală: $64@55 \times 55$.

Exercițiul R.2: Calculul Parametrilor (Convoluția 1×1 în ResNet)

Considerați un bloc de proiecție ResNet în care se dorește dublarea canalelor de la $C_{in} = 128$ la $C_{out} = 256$, reducând în același timp rezoluția spațială (folosind Stride 2).

- Convoluția 1×1 pe *shortcut*: $C_{in} = 128, C_{out} = 256, k = 1 \times 1$, Stride 2.

1. Calculați numărul total de parametri reali (Greutăți V și Bias-uri u) pentru această convoluție 1×1.
2. Dacă rezoluția intrării era 32×32, care este rezoluția spațială a ieșirii după această convoluție 1×1 cu Stride 2?

Rezolvare R.2:

1. **Numărul Total de Parametri:**

$$\text{Greutăți } V = C_{out} \times C_{in} \times k_h \times k_w = 256 \times 128 \times 1 \times 1 = 32,768$$

$$\text{Bias-uri } u = C_{out} = 256$$

$$\text{Total Parametri} = 32,768 + 256 = \mathbf{33,024}$$

2. **Rezoluția Spațială a Ieșirii:**

$$n'_h = \lfloor (n_h - k_h + p_h)/s_h \rfloor + 1$$

Asumând Padding 0, Stride $s = 2, k = 1$:

$$n'_h = \lfloor (32 - 1 + 0)/2 \rfloor + 1 = \lfloor 31/2 \rfloor + 1 = 15 + 1 = 16$$

Rezoluția spațială este $\mathbf{16} \times \mathbf{16}$.

Exercițiul R.3: Verificarea Mapării Reziduale (Numeric)

Considerați un bloc rezidual simplificat (Fără BN sau ReLU, doar două straturi de ponderi W_1, W_2).

$$f(\mathbf{x}) = W_2(W_1\mathbf{x})$$

Dacă se dorește ca maparea rezultată să fie exact funcția identitate $f(\mathbf{x}) = \mathbf{x}$.

1. Ce funcție (reziduu) ar trebui să învețe blocul intern $W_2(W_1\mathbf{x})$?
2. Dacă $\mathbf{x} = 5$ (scalar), și $W_1 = 0.5, W_2 = 2.0$ (fără bias), ce valoare are reziduul învățat?

Rezolvare R.3:

1. **Funcția Reziduală Dorită:** Blocul intern învață maparea reziduală $R(\mathbf{x}) = f(\mathbf{x}) - \mathbf{x}$. Dacă $f(\mathbf{x}) = \mathbf{x}$, atunci $R(\mathbf{x}) = \mathbf{x} - \mathbf{x} = \mathbf{0}$. Blocul intern trebuie să învețe **maparea zero**.

2. **Valoarea Reziduului Învățat (cu $W_1 = 0.5, W_2 = 2.0$):** Calculăm maparea standard: $f(\mathbf{x}) = W_2(W_1\mathbf{x}) = 2.0 \cdot (0.5 \cdot 5) = 2.0 \cdot 2.5 = 5.0$. Maparea reziduală: $R(\mathbf{x}) = f(\mathbf{x}) - \mathbf{x} = 5.0 - 5 = \mathbf{0}$. (Acest exemplu simplificat arată că, dacă ponderile sunt setate astfel încât $W_2W_1 = 1$ (sau o matrice identitate generalizată), maparea reziduală este exact 0, demonstrând că funcția identitate este ușor de obținut).

3.11

Ați parcurs o secțiune extrem de densă și fundamentală a cursului despre Rețele Neuronale Convoluționale (CNN). Excelent! Pentru a consolida înțelegerea fiecărui subiect cheie, voi genera un set de exerciții de calcul și analiză, bazându-mă pe structura și complexitatea demonstrate în sursa *03_11_Exercises.pdf*.

Exerciții de Consolidare CNN (Convoluții, Arhitecturi și Normalizare)

Secțiunea I: Convoluții Fundamentale și Parametrizare

Exercițiul I.1: Calculul Parametrilor și Dimensiunea Ieșirii

(Acest exercițiu este similar cu structura din sursa, dar cu parametri modificați.)

Definiți un strat convoluțional cu:

- Canale de intrare $C_{in} = 4$.
- Canale de ieșire $C_{out} = 3$.
- Dimensiunea kernel-ului $k_h \times k_w = 3 \times 3$.
- Intrarea X are dimensiunea spațială $n_h \times n_w = 6 \times 6$.
- Fără padding și stride 1.

1. **Calculați numărul total de parametri reali** (greutăți V și bias-uri u) pentru strat.
2. **Calculați dimensiunea spațială a ieșirii** $n'_h \times n'_w$.
3. **Specificați dimensiunea tensorului** V (Kernelului) și dimensiunea vectorului u (Bias-ul).

Rezolvare I.1:

1. **Numărul Total de Parametri:**

$$\text{Număr Greutăți } V = C_{out} \times C_{in} \times k_h \times k_w = 3 \times 4 \times 3 \times 3 = \mathbf{108}$$

$$\text{Număr Bias-uri } u = C_{out} = \mathbf{3}$$

$$\text{Total parametri} = 108 + 3 = \mathbf{111}$$

2. **Dimensiunea Spațială a Ieșirii:**

$$n'_h \times n'_w = (n_h - k_h + 1) \times (n_w - k_w + 1)$$
$$n'_h \times n'_w = (6 - 3 + 1) \times (6 - 3 + 1) = 4 \times 4$$

   Dimensiunea ieșirii este $\mathbf{4} \times \mathbf{4}$.

3. **Dimensiunea Tenzorului V și u:**
   - $V \in \mathbb{R}^{C_{out} \times C_{in} \times k_h \times k_w} \to \mathbf{V} \in \mathbb{R}^{\mathbf{3} \times \mathbf{4} \times \mathbf{3} \times \mathbf{3}}$
   - $u \in \mathbb{R}^{C_{out} \times 1} \to \mathbf{u} \in \mathbb{R}^{\mathbf{3} \times \mathbf{1}}$

Exercițiul I.2: Cross-Correlation pe Canale Multiple (Calcul Punctual)

Calculați elementul de ieșire $\mathbf{H}_{0,0,1}$ (primul rând, prima coloană, primul canal de ieșire) al unui strat convoluțional, dat fiind următorii parametri:

- $C_{in} = 2, C_{out} = 1, k_h \times k_w = 2 \times 2.$
- Bias-ul canalului de ieșire 1 este $u_1 = 2.$
- Intrări (patch 2×2 centrat pe (0, 0)): $X_{c=1} = [2 \quad 1\ 0 \quad 3], X_{c=2} = [1 \quad 0\ 0 \quad 1]$
- Kernel (pentru Canalul de Ieșire 1): $V_{c=1,d=1} = [1 \quad 1\ 1 \quad 1], V_{c=2,d=1} = [2 \quad 0\ 0 \quad 2]$

Formula (conform sursei): $[\mathbf{H}]_{i,j,d} = [\mathbf{u}]_d + \sum_c \left(\sum_{a,b} [\mathbf{V}]_{a,b,c,d} [\mathbf{X}]_{i+a,j+b,c} \right)$

Rezolvare I.2:

1. **Contribuția din Canalul de Intrare 1 ($c = 1$):**

$$\text{Suma}_1 = (1 \cdot 2) + (1 \cdot 1) + (1 \cdot 0) + (1 \cdot 3) = 2 + 1 + 0 + 3 = 6$$

2. **Contribuția din Canalul de Intrare 2 ($c = 2$):**

$$\text{Suma}_2 = (2 \cdot 1) + (0 \cdot 0) + (0 \cdot 0) + (2 \cdot 1) = 2 + 0 + 0 + 2 = 4$$

3. **Ieșirea Finală $\mathbf{H}_{0,0,1}$:**

$$[\mathbf{H}]_{0,0,1} = u_1 + \text{Suma}_1 + \text{Suma}_2 = 2 + 6 + 4 = \mathbf{12}$$

Secțiunea II: Padding, Stride și Pooling

Exercițiul II.1: Controlul Dimensionalității (Padding și Stride)

O rețea primește o imagine 50×50 cu 3 canale. Se aplică un strat convoluțional cu:

- Kernel $k_h \times k_w = 5 \times 5.$
- Stride $s_h \times s_w = 2 \times 2.$

- Padding (total) $p_h \times p_w = 2 \times 2$.

Calculați dimensiunea spațială a ieșirii ($n'_h \times n'_w$).

Formula dimensională: $n' = \lfloor (n - k + p)/s \rfloor + 1$

Rezolvare II.1:

1. **Calculul Înălțimii Ieșirii (n'_h):**

$$n'_h = \lfloor (50 - 5 + 2)/2 \rfloor + 1$$
$$n'_h = \lfloor 47/2 \rfloor + 1 = 23 + 1 = \mathbf{24}$$

2. **Calculul Lățimii Ieșirii (n'_w):**

$$n'_w = 24$$

   Dimensiunea ieșirii este $\mathbf{24} \times \mathbf{24}$.

Exercițiul II.2: Max-Pooling și Average-Pooling

Considerați un tensor de intrare X și aplicați o operație de **Maximum Pooling** și una de **Average Pooling**, ambele cu o fereastră de 2×2 și Stride 2.

$$X = [8 \quad 1 \quad 4 \quad 5\ 2 \quad 9 \quad 3 \quad 6\ 1 \quad 1 \quad 1 \quad 1\ 4 \quad 4 \quad 4 \quad 4]$$

1. Calculați ieșirea Y_{Max} (Max Pooling).
2. Calculați ieșirea Y_{Avg} (Average Pooling).

Rezolvare II.2:

1. **Max Pooling (Y_{Max}):**
   - Fereastra 1: $\max(8, 1, 2, 9) = 9$.
   - Fereastra 2: $\max(4, 5, 3, 6) = 6$.
   - Fereastra 3: $\max(1, 1, 4, 4) = 4$.
   - Fereastra 4: $\max(1, 1, 4, 4) = 4$.

$$Y_{\text{Max}} = [9 \quad 6\ 4 \quad 4]$$

2. **Average Pooling (Y_{Avg}):**
   - Fereastra 1: $(8 + 1 + 2 + 9)/4 = 20/4 = 5$.
   - Fereastra 2: $(4 + 5 + 3 + 6)/4 = 18/4 = 4.5$.
   - Fereastra 3: $(1 + 1 + 4 + 4)/4 = 10/4 = 2.5$.
   - Fereastra 4: $(1 + 1 + 4 + 4)/4 = 10/4 = 2.5$.

$$Y_{\text{Avg}} = [5.0 \quad 4.5\ 2.5 \quad 2.5]$$

Secțiunea III: Arhitecturi Avansate (VGG, BN, ResNet)

Exercițiul III.1: Urmărirea Fluxului Dimensional în Blocul VGG

Considerați un modul standard VGG care utilizează un strat convoluțional 3×3 cu Padding 1 și Stride 1, urmat de Maximum Pooling 2×2 cu Stride 2.

- Intrare: 16×16, 128 de canale.

1. Calculați dimensiunea ieșirii după stratul convoluțional (asumând $C_{out} = 256$).
2. Calculați dimensiunea ieșirii după stratul de Maximum Pooling.
3. Calculați numărul total de parametri (Greutăți și Bias) pentru stratul convoluțional (Conv 3×3, $C_{in} = 128, C_{out} = 256$).

Rezolvare III.1:

1. **După Stratul Convoluțional (VGG Block):**
   - Padding 1 și kernel 3×3 mențin dimensiunea spațială: 16×16.
   - Dimensiune ieșire: $\mathbf{256@16 \times 16}$.
2. **După Stratul de Maximum Pooling:**
   - Pooling 2×2 cu Stride 2 înjumătățește rezoluția spațială: $16/2 = 8$.
   - Numărul de canale rămâne neschimbat (256).
   - Dimensiune ieșire: $\mathbf{256@8 \times 8}$.
3. **Parametrii Stratului Convoluțional:**

$$\text{Greutăți } V = 256 \times 128 \times 3 \times 3 = 256 \times 1152 = \mathbf{294,912}$$

$$\text{Bias-uri } u = 256$$

$$\text{Total parametri} = 294,912 + 256 = \mathbf{295,168}$$

Exercițiul III.2: Analiza Normalizării pe Mini-Batch (BN)

Un strat FC primește un mini-batch de 4 exemple, cu o singură unitate ascunsă:

$$\mathcal{B} = x_1 = 2, x_2 = 4, x_3 = 6, x_4 = 10$$

1. Calculați media eșantionului $\hat{\mu}_{\mathcal{B}}$.
2. Calculați deviația standard $\hat{\sigma}_{\mathcal{B}}$ (presupuneți $\epsilon = 0$ și folosiți formula pentru varianță $\hat{\sigma}^2_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2$).

Rezolvare III.2:

1. **Calculul Mediei ($\hat{\mu}_{\mathcal{B}}$):**

$$\hat{\mu}_{\mathcal{B}} = (2 + 4 + 6 + 10)/4 = 22/4 = \mathbf{5.5}$$

2. **Calculul Varianței ($\hat{\sigma}_{\mathcal{B}}^2$):**

$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{4}\left[(2-5.5)^2 + (4-5.5)^2 + (6-5.5)^2 + (10-5.5)^2\right]$$

$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{4}\left[(-3.5)^2 + (-1.5)^2 + (0.5)^2 + (4.5)^2\right]$$

$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{4}[12.25 + 2.25 + 0.25 + 20.25] = 35/4 = 8.75$$

Deviația standard ($\hat{\sigma}_{\mathcal{B}}$): $\sqrt{8.75} \approx \mathbf{2.958}$

Exercițiul III.3: Funcția de Identitate în ResNet

Explicați matematic de ce blocul rezidual facilitează învățarea funcției identitate ($f(\mathbf{x}) = \mathbf{x}$) mai ușor decât un bloc standard, utilizând conceptul de *reziduu*.

Răspuns III.3:

Blocul standard învață direct maparea $f(\mathbf{x})$. Pentru a obține funcția identitate, ponderile stratului ar trebui să fie inițializate și antrenate astfel încât să se apropie de o matrice identitate, ceea ce este dificil și instabil pentru optimizare.

Blocul rezidual învață maparea reziduală $R(\mathbf{x}) = f(\mathbf{x}) - \mathbf{x}$. Dacă maparea optimă este funcția identitate, atunci $f(\mathbf{x}) = \mathbf{x}$, ceea ce implică faptul că **reziduul optim dorit este** $R(\mathbf{x}) = 0$.

Este mult mai ușor pentru un optimizator să forțeze ponderile și bias-urile straturilor convoluționale din blocul rezidual (ex: W_1, W_2) la valori foarte mici (aproape zero), deoarece zero se află în interiorul regiunii de valori mici din spațiul de parametrii, asigurând că $R(\mathbf{x}) \approx 0$. Prin conexiunea *shortcut* aditivă, ieșirea devine $\mathbf{y} = \mathbf{x} + R(\mathbf{x}) \approx \mathbf{x}$, realizând funcția identitate .

Exercițiul III.4: Convoluția 1×1 în ResNet (Tip Proiecție)

O rețea ResNet necesită un strat de proiecție (bloc 1×1 Conv pe *shortcut*) pentru a schimba numărul de canale de la 64 la 128.

1. Câte greutăți are acest strat 1×1?
2. Dacă acest strat 1×1 ar avea un *stride* de 2 (fără padding), care ar fi scopul său dual?

Rezolvare III.4:

1. **Numărul de Greutăți:**
   - $C_{in} = 64, C_{out} = 128, k = 1 \times 1.$
   - $$\text{Greutăți } V = C_{out} \times C_{in} \times 1 \times 1 = 128 \times 64 = \mathbf{8192}$$
2. **Scopul Dual (Stride 2):** Stratul 1×1 convoluțional cu *stride* 2 servește două scopuri simultane: a) **Ajustarea Canalelor:** Transformă $C_{in} = 64$ în $C_{out} = 128$ (schimbând

adâncimea). b) **Ajustarea Rezoluției Spațiale:** *Stride*-ul 2 aplică *downsampling* (înjumătățește înălțimea și lățimea), asigurând că intrarea $\mathbf{x}$ are aceeași formă spațială ca ieșirea celorlalte straturi din blocul rezidual, permițând operația de adunare element-wise.